A Traffic Management System for Large and Heterogeneous Vehicles in Narrow Industrial Environments

International Journal of Robotics Research
XX(X):1–30
©The Author(s) 2025
Reprints and permission:
sagepub.co.uk/journalsPermissions.nav
DOI: 10.1177/ToBeAssigned
www.sagepub.com/
SAGE

Alessandro Bonetti[1], Silvia Proia[1], Simone Guidetti[2], and Lorenzo Sabattini[1]

Abstract

The coordination of Automated Guided Vehicles (AGVs) in high-density industrial environments represents a critical challenge within Logistics 4.0, as traditional traffic management methods often lead to inefficiencies caused by negotiation-based priority assignment. To overcome the resulting limitations, this paper presents an innovative AGV traffic management system based on a Lifelong Multi-Agent Path Finding (L-MAPF) algorithm operating on roadmaps generated with Non-Uniform Rational B-Splines (NURBS) curves. The approach guarantees locally optimal coordination and ensures safe operation of large and heterogeneous AGVs. Building on this concept, the proposed framework integrates a modified version of the Bounded Horizon Conflict Based Search (CBS) technique within a Rolling Horizon Conflict Resolution strategy, utilizing an extended time horizon for each agent to enable effective conflict resolution in corridors identified by a topological map. In contrast to state-of-the-art methods for AGV fleet traffic management, the proposed solution is designed for real-world, non-standardized (i.e., non grid-like) industrial settings characterized by narrow bidirectional corridors and high-traffic density, where AGVs of various sizes and capabilities operate simultaneously. Key contributions include an anytime conflict resolution strategy, dynamic corridor expansion, adaptive time horizon regulation, and an advanced mechanism for deadlock detection and resolution. Experimental results obtained in realistic industrial environments demonstrate higher throughput, with improvements of up to 11% over a conventional rule-based traffic management system, a state-of-the-art industrial method, and a priority-based L-MAPF variant, while maintaining continuous operation and improved efficiency.

Keywords

Multi-AGV System, Traffic Management, Lifelong Multi-Agent Path Finding, Conflict-Based Search, Deadlock.

1 Introduction

In the Logistics 4.0 paradigm (Winkelhaus and Grosse 2020), which aims at creating intelligent, interoperable, and autonomous environments, the problem of managing and controlling fleets of mobile robots is attracting enormous interest (Proia et al. 2022; Azadeh et al. 2019).

In order to develop an automated industrial system, Automated Guided Vehicles (AGVs) play a crucial role due to their ability to optimize material handling tasks, improve inventory management, enhance safety, and increase operational efficiency (Siegfried and Bourafa 2023).

Effective AGV operation requires addressing two primary concerns: task assignment and traffic management (Santos et al. 2021). On the one hand, task assignment consists of dynamically assigning AGVs to specific missions, such as transporting goods between different locations, while considering factors like vehicle availability, task urgency, and overall system efficiency. On the other hand, traffic management focuses on generating safe and efficient paths for AGVs and coordinating their movement throughout the environment to ensure smooth operation. Key aspects include establishing the right-of-way and managing AGV interactions in shared spaces, with the objective of preventing collisions and deadlocks while minimizing downtime.

However, to boost productivity and operational effectiveness, traditional rule-based traffic management systems are no longer sufficient to meet the growing demands of modern logistics operations. Specifically, rule-based coordination on AGV paths can lead to inefficiencies, such as bottlenecks and inappropriate priority handling, reducing overall efficiency (Pratissoli et al. 2023). To overcome the above limitations, research and industrial sectors are increasingly turning to more advanced, data-driven traffic management solutions that can adapt in real-time to changing conditions and improve performance without relying on fixed rules (De Ryck et al. 2020). In this context, the application of Multi-Agent Path Finding (MAPF) algorithms is essential to facilitate the navigation of AGVs (Wagner and Choset 2015; Dergachev and Yakovlev 2021; Okumura et al. 2022). The MAPF problem, a well-established combinatorial challenge in robotics and artificial intelligence, focuses on identifying collision-free trajectories for agents with predetermined start and goal locations within a specified environment (Zhang et al. 2024). Traditional MAPF approaches primarily address a static, "one-shot" version of the problem. However, to handle dynamic scenarios where tasks change over time and

[1]Department of Sciences and Methods for Engineering (DISMI), University of Modena and Reggio Emilia, Italy.
[2]Gruppo TecnoFerrari S.p.a. con socio unico, Italy.

Corresponding author:
Silvia Proia, Department of Sciences and Methods for Engineering (DISMI), University of Modena and Reggio Emilia, Italy
Email: silvia.proia@unimore.it

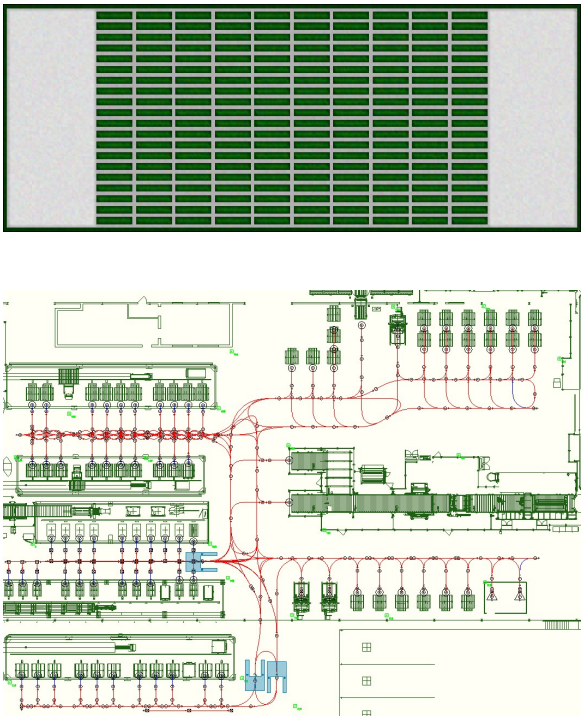


Figure 1. The figure illustrates a standardized warehouse environment modeled as a gridmap of vehicle-sized cells (Stern et al. 2019) (a) compared to a non-standardized complex layout, which depicts a plant of our use case derived from a real application (b). Specifically, (b) presents the plant configuration, where green lines represent stations, machinery, battery chargers, and other relevant features within the layout, red lines indicate the roadmap, and blue shapes represent AGV footprints.

where operating conditions are affected by uncertainties such as vehicle alarms, manual interventions, or safety scanner activations, the Lifelong MAPF (L-MAPF) framework is introduced. Specifically, the lifelong approach continuously resolves MAPF instances in real-time, accounting for a series of tasks throughout the operational lifespan of the agents (Li et al. 2021). By coordinating the simultaneous movement of multiple AGVs, L-MAPF algorithms ideally prevent collisions and deadlocks in dynamic environments.

Nevertheless, the majority of L-MAPF algorithms have been applied to standardized, i.e., grid-like, environments (Song et al. 2023; Stern et al. 2019) (see Fig.1a), and rely on assumptions that may limit their applicability to more complex systems typically encountered in industrial settings (Gao et al. 2023), such as homogeneous vehicles and unit-time actions. Additionally, the inherent constraints of real-world implementations, coupled with unpredictable conditions, can still lead to deadlocks (Małopolski and Zajac 2021). Therefore, to effectively mitigate the above issues, even the most advanced L-MAPF systems must integrate mechanisms for detecting and resolving deadlocks, ensuring conflict-free and continuous AGV operation.

In this paper, we address the challenges of traffic management for large and heterogeneous AGVs operating in complex, non-standardized industrial environments (see Fig.1b). Although we present a complete system, our primary focus is on fleet coordination. Specifically, we propose a novel coordination strategy based on an L-MAPF algorithm designed to meet the demands of real-world scenarios, ensuring efficient and reliable AGV coordination in dynamic high-traffic settings. Additionally, we introduce a cutting-edge strategy for deadlock detection and resolution that ensures continuous operational flow. Extensive testing in both simulations and real-world environments demonstrates real-time capability of the proposed solution and significant improvements in throughput compared to traditional rule-based coordination.

The remainder of this paper is structured as follows. Section 2 summarizes the related works and sheds light on the main contributions of this work, positioning them with respect to the state-of-the-art. Section 3 presents the problem statement and introduces key concepts from graph theory used to model the environment, as well as the fundamental notions required to represent the AGVs, together with the related assumptions. The architecture of the proposed traffic management system is then presented in Section 4. The environment model, organized into two distinct layers, is described in Section 5. Section 6 details the path generation and the trajectory definition. The proposed methodology, including the mathematical formalization of the L-MAPF coordinator, is provided in Section 7. Section 8 illustrates the path allocator, which ensures the safe execution of the coordinated trajectories, while Section 9 discusses the deadlock detection and resolution module. The experimental setup and the corresponding results are presented in Section 10. Finally, concluding remarks are provided in Section 11.

2 Related Works and Paper Contributions

2.1 Related Works

In the scientific and industrial fields, the efficient coordination of high-density AGV fleets in irregular and non-standardized industrial settings presents significant challenges due to the constrained solution space and limited system configurations available to ensure both optimal and continuous operation while avoiding collisions and deadlocks. Typically, in real AGV systems, large and heterogeneous vehicles are restricted to follow a predefined set of virtual or physical paths, known as a roadmap. This is particularly necessary for large, heavy vehicles operating in confined or irregular spaces, such as narrow corridors, where safety and robustness must be ensured.

Over the past decade, a variety of studies have emerged in the literature concerning the coordination of AGV fleets, which are generally classified into two main categories based on their solvers: rule-based and search-based methods (see Table 1) (Gao et al. 2023).

Rule-based coordination builds upon and enhances the traditional concept of industrial traffic management systems by utilizing traffic and negotiation rules to assign priority and thus to solve conflicts between vehicles. For instance, in Digani et al. (2015), conflicts among AGVs are managed using a hybrid approach that integrates a priority-based negotiation mechanism with a resource allocation strategy, where resources are defined as intersection areas within a structured and organized industrial layout. Despite simulations being conducted with up to thirty AGVs, vehicles are homogeneous and the coordination is performed within fairly standardized settings without providing any

Table 1. Summary of works related to coordination strategies for AGV traffic management.

Algorithm Type	Ref. No	Max Simulated Vehicles	Graph	Local Optimality	Anytime Strategy	Large & Heterogeneous Agents	Non-Standardized Environment	High-Traffic Density	Real Implementation
Rule-based	Digani et al. (2015)	30	Topological map + Roadmap	✗	✗	✗	✗	✗	✗
	Verma et al. (2024)	10	Roadmap	✗	✗	✗	✗	✓	✗
	Pratissoli et al. (2023)	8	Topological map + Roadmap	✗	✗	✗	✓	✓	✓
	Hu et al. (2021)	3	Roadmap	✗	✗	✓	✗	✗	✗
Search-based	Chen et al. (2023b)	20	Gridmap	✗	✗	✗	✗	✗	✓
	Xie et al. (2024)	20	Gridmap	✗	✗	✗	✗	✗	✗
	Liu et al. (2021)	1008	Topological map + Roadmap	✓	✗	✗	✗	✓	✗
	Song et al. (2023)	150	Topological map + Gridmap	✓	✓	✗	✗	✓	✗
	ours	20	Topological map + Roadmap	✓	✓	✓	✓	✓	✓

formal guarantee of optimality. The experiments are also limited to low-traffic conditions, with no more than four vehicles per sector.

An alternative method, proposed by Verma et al. (2024), presents an improved dynamic resource reservation that exploits multiple dynamic reservations of shared resource points combined with a set of traffic rules that detects, classifies, and solves conflicts as they eventually arise. However, such rules lack general applicability, and the achieved coordination relies on heuristics. Furthermore, the approach has only been tested in highly simplified environments with homogeneous vehicles and has not yet been implemented in real-world scenarios.

To overcome the above limitations, Pratissoli et al. (2023) introduces a Coordination Diagram to solve conflicts by establishing negotiation rules for each timestep within a given time horizon. The proposed space-time approach enables the application of simpler rules that can be easily adapted to real-world, non-standardized, high-traffic environments. However, this coordination strategy has only been tested with eight homogeneous vehicles and does not guarantee any degree of optimality.

A more adaptive approach is illustrated in Hu et al. (2021), where the authors combine Behavior Trees with Reinforcement Learning to adaptively choose the most effective rule-based strategy from a set of available optional strategies according to diverse, dynamic, and complex situations in Industry 4.0 settings. Despite its potential, this study is limited to a specific and simplified environment involving only three operational vehicles, indicating that further developments are necessary for effective AGV coordination in real industrial scenarios.

Therefore, rule-based solvers exhibit high-speed computation and are capable of addressing medium-scale problems, but they provide no formal performance guarantees (Gao et al. 2023). They also require substantial expert configuration to ensure consistent coordination across different industrial plants.

On the other hand, search-based methods eliminate the need for predefined rules and employ algorithms to methodically address the challenge of vehicle coordination. For instance, in Chen et al. (2023b), a Coloured Petri Net is employed to coordinate a fleet of AGVs, effectively eliminating active deadlocks and preventing passive ones. This approach has been successfully simulated with up to twenty vehicles and tested in real facilities. However, it does not guarantee optimality and is limited to standardized environments modeled as gridmaps.

Building on the existing literature, Xie et al. (2024) introduces a centralized planner based on an improved conflict-free A* algorithm for AGV fleet coordination. The proposed algorithm successfully manages the coordination of twenty vehicles, but it does not guarantee optimality and operates solely within a standardized environment that do not reflect real industrial constraints.

For achieving optimal navigation of AGVs, traffic management of AGV fleets must rely on algorithms specifically designed to address the L-MAPF problem. One remarkable example, presented in Liu et al. (2021) has effectively applied this methodology to an AGV coordination framework. In particular, the work employs a hierarchical approach to predict traffic flow within sectors of a topological map and to plan vehicle paths accordingly. Conflicts are resolved in a decentralized manner through the use of Conflict Based Search (CBS) and Cooperative A* algorithms, enabling the coordination of a large number of agents, potentially up to a thousand, in a locally optimal manner. However, the considered fleet is homogeneous, and the environment remains highly standardized, consisting of sectors featuring a single intersection connected by unidirectional lanes without intersecting stations. Each sector contains a roadmap represented as a fully discretized graph composed of unit-time segments, where both intersections and lane portions are modeled as edges with uniform traversal duration. Furthermore, a significant simplifying assumption is introduced: "for each new task, the pick-up station and its corresponding working station must differ from those of any unaccomplished tasks already assigned". In numerous industrial settings, particularly in operations involving palletizers and pallet wrappers, it is common practice to install fewer machines than the number of AGVs in operation. This is due to the high operating speeds of the machines and the imperative to minimize costs. Consequently, coordinating two or more AGVs is often necessary to serve the same working station, ensuring a smooth and efficient workflow. As a result, this method struggles to coordinate vehicles in irregular spaces with narrow, bidirectional corridors and fails to efficiently manage situations where multiple vehicles need to service the same destination station.

Building on the Rolling Horizon Conflict Resolution (RHCR) framework (Li et al. 2021), Song et al. (2023) introduces an Anytime L-MAPF algorithm on topological maps. The approach first computes locally bounded sub-optimal solutions using Enhanced CBS (ECBS) on a gridmap and then refines them iteratively by applying CBS to the topological map under a fixed computation timeout. In this manner, fast initial solutions are generated and progressively improved over time, enhancing scalability under high-density conditions. However, the proposed

framework achieves higher coordination efficiency only in highly standardized environments composed exclusively of corridors, and it further relies on the simplifying assumption that each action has unit time duration. Moreover, the evaluation is limited to simulation, where up to one hundred fifty homogeneous agents are coordinated through a periodic replanning interval of ten seconds. Such an update cycle remains incompatible with the responsiveness required in dynamic industrial operations characterized by frequent task reassignment, human intervention, and uncertainty. Although the method provides a mechanism for gradual refinement of sub-optimal solutions, it still lacks real-time adaptability and neglects heterogeneity in kinematic constraints and motion capabilities.

Hence, search-based methods can mitigate deadlock formation and can theoretically attain optimal performance using L-MAPF algorithms. However, current solutions in the literature fall short in effectively coordinating fleets of large and heterogeneous AGVs within non-standardized environments characterized by narrow, bidirectional corridors, columns, and other irregularities, particularly under conditions of high-traffic density. Many existing approaches rely on strong assumptions that are not typically applicable to complex industrial settings.

More broadly, existing L-MAPF algorithms exhibit several structural limitations, including the common assumption of homogeneous agents and unit-time actions (Li et al. 2021; Madar et al. 2022; Xu et al. 2022; Ma et al. 2017; Damani et al. 2021). Therefore, most formulations operate on gridmaps, where each cell represents a vehicle-sized unit, which considerably constrains the operational space for larger AGVs such as forklifts. Such discretization often leads to inefficiencies and limits applicability in non-standardized industrial environments characterized by narrow corridors, irregular geometries, and obstacles including columns or machinery. The underlying assumptions, while simplifying theoretical analysis and ensuring formal completeness, ultimately restrict the use of L-MAPF algorithms in real-world coordination problems involving heterogeneous fleets and complex, continuous roadmaps. A notable exception is the work of Andreychuk et al. (2022), which introduces a continuous-time MAPF formulation that removes the above assumptions by allowing agents to follow continuous trajectories in both space and time, without relying on discrete grid representations or unit-timesteps. Although the formulation represents a significant advancement toward bridging discrete MAPF and real-world motion, it is not intended for lifelong scenarios and therefore fails to account for dynamically changing environments or sequential task assignments that characterize AGV coordination in industrial settings. In addition, the method remains computationally demanding, which further limits its applicability under real-time constraints.

2.2 Contributions

As illustrated in Section 2.1, the literature lacks solutions capable of coordinating large and heterogeneous AGVs in non-standardized, high-traffic environments, e.g., the scenario depicted in Fig. 1b, while ensuring locally optimal performance. On the one hand, rule-based methods are capable of effectively addressing problems of up to medium scale. However, their reliance on static and unyielding rules prevents them from ensuring general applicability or providing any formal performance guarantees. This limitation is particularly critical for enhancing the performance of traditional AGV traffic management systems. Improving their efficiency is crucial for increasing the overall productivity of plants and warehouses, which is the primary objective of our study. Additionally, the majority of works either rely on oversimplifications or lack real-world implementation, making them unsuitable for use in operational facilities. On the other hand, search-based methods can systematically coordinate fleets of AGVs and provide guarantees on solution quality through L-MAPF algorithms. However, applicability remains limited in real-world environments, as most methods rely on restrictive assumptions such as homogeneous agents and unit-time action durations. The resulting simplifying assumptions, together with grid-based environment representations, make theoretical analysis feasible but do not capture the complexity of industrial scenarios involving large, heterogeneous vehicles operating on continuous roadmaps within non-standardized environments. Consequently, existing search-based approaches cannot be directly applied to the case study under consideration.

Motivated by the above considerations, this paper extends our previous work (Bonetti et al. 2024b) and introduces an integrated AGV traffic management system tailored to real industrial environments. The objective is to achieve locally optimal coordination for large, heterogeneous fleets operating on complex roadmaps with non-uniform traversal times and geometric properties. The system addresses the shortcomings of rule-based approaches, which lack performance guarantees, and search-based methods, which rely on restrictive assumptions. By removing the resulting simplifications, the proposed solution closes the gap between theoretical advancements and practical AGV operations, enabling the deployment of advanced coordination techniques while ensuring safety, efficiency, and continuous operation in real facilities.

Our system utilizes an L-MAPF algorithm on roadmaps constructed with Non-Uniform Rational B-Splines (NURBS) curves, ensuring continuity and compliance with the AGVs' non-holonomic constraints. In particular, we integrate a modified version of the Bounded Horizon CBS technique within the RHCR framework. By utilizing an extended time horizon for each individual agent, this adaptation enables effective conflict resolution in corridors identified by a topological map.

Unlike our previous work (Bonetti et al. 2024b), which, to the best of the authors' knowledge, is the only one in the related literature focusing on coordinating AGVs with guaranteed locally optimal performance in real industrial environments, this study incorporates key changes and innovations. Here, we significantly extend the approach by:

i) proposing a new environment model that enables heterogeneous AGV operation and better captures the geometric characteristics of spatial interactions, ensuring precise collision detection in confined areas without compromising computational efficiency;

ii) implementing an anytime strategy to ensure real-time performance while progressively refining solutions toward eventual optimality and completeness, thereby

guaranteeing that feasible solutions are produced within strict computational time constraints and can continually improve until converging to optimal and complete outcomes;
iii) introducing a path allocator module that formalizes the execution layer of the traffic management system and ensures safe, standard-compliant interaction with real AGVs. The module bridges the discrete L-MAPF trajectories with the continuous and uncertain motion of industrial vehicles, enabling reliable deployment in operational environments;
iv) presenting a novel deadlock detection and resolution strategy to enhance the system's robustness, particularly in dynamic environments where tasks may change unexpectedly. This strategy is based on another of our works (Bonetti et al. 2024a) and further elaborated upon in the current manuscript. Our approach ensures real-time adaptability, allowing AGVs to efficiently navigate high-traffic areas and minimize operational interruptions;
v) validating the approach in real-world facilities and benchmarking it against a traditional rule-based traffic management system employed by logistics companies, a state-of-the-art industrial solution, and a comparative L-MAPF method. The results demonstrate superior performance in terms of throughput, optimality of the proposed coordination framework, and effectiveness of the deadlock detection and resolution strategy. Additionally, our solution consistently achieves real-time performance in high-traffic, cluttered environments and adapts well to medium-scale and standardized settings, offering flexibility across various logistics and production scenarios.

3 Preliminaries

In this section, we outline the problem statement (Section 3.1), the key concepts to model the environment (Section 3.2), and assumptions that ensure the feasibility of the proposed approach (Section 3.3).

3.1 Problem Statement

We consider a non-standardized industrial environment where a fleet of large and heterogeneous AGVs operates along a roadmap that captures the geometric and operational constraints of the facility, including narrow bidirectional corridors, intersections, and irregular layouts. The vehicles differ in size, kinematic behavior, and motion capabilities, and are required to perform a sequence of pick-up and drop-off operations within a shared workspace.

The objective of the traffic management system is to generate coordinated motion plans that enable all vehicles to navigate safely and efficiently while completing their assigned tasks. The problem integrates path planning and fleet coordination into a unified framework that ensures collision-free and dynamically feasible motion, while simultaneously detecting and resolving deadlocks and minimizing overall traversal cost in terms of throughput and travel time. Safe operation must be guaranteed by avoiding both spatial and temporal conflicts, maintaining continuous traffic flow even under high-density traffic conditions.

Industrial environments are inherently dynamic, as tasks may be reassigned, temporary obstacles can appear, and operational priorities may change during execution. The coordination framework must therefore operate in real-time, continuously adapting vehicle trajectories to the evolving system state, variations in task assignment or workspace configuration, and situations that may result in deadlocks such as blocked corridors or mutual agent waiting.

3.2 Definitions

To better understand the different steps of the proposed approach, we introduce the following definitions, grouped into environment-related (Bondy and Murty 2008) and AGV-related concepts.

We first introduce the definitions related to the representation of the environment, which formalize the roadmap structure and its topological properties.

Definition 1. Directed Graph. *A directed graph $\mathcal{G}$ is characterized by a set $\mathcal{V}(\mathcal{G})$ of vertices or nodes and a set $\mathcal{E}(\mathcal{G}) \subseteq \mathcal{V}(\mathcal{G}) \times \mathcal{V}(\mathcal{G})$ of edges.*

Definition 2. Subgraph. *A subgraph $\mathcal{K}$ of $\mathcal{G}$ is a graph whose vertex set $\mathcal{V}(\mathcal{K})$ and edge set $\mathcal{E}(\mathcal{K})$ are subsets of $\mathcal{V}(\mathcal{G})$ and $\mathcal{E}(\mathcal{G})$, respectively.*

Definition 3. Path. *A path of length L is defined on $\mathcal{G}$ as an ordered sequence of vertices $\{v_1, v_2, \dots, v_L\} \subseteq \mathcal{V}(\mathcal{G})$ such that an edge exists in $\mathcal{E}(\mathcal{G})$ from vertex v_i to vertex v_{i+1}, for $i \in \{1, \dots, L-1\}$.*

Definition 4. Graph Connectivity. *A graph $\mathcal{G}$ is said to be connected if a path exists between each pair of vertex in $\mathcal{V}(\mathcal{G})$. Given any pair of vertices v^m, v^n, a directed graph is considered strongly connected if a directed path can be defined from v^m to v^n and vice versa.*

Definition 5. Minimum Cost Path. *Given a graph $\mathcal{G}$, the minimum cost path between vertex v_i and vertex v_{i+1} is determined by the edge weights $\omega_{i,i+1}$.*

Definition 6. Location. *A location p corresponds to a specific pose a vehicle can assume within the environment, expressed as $q = [x, y, \theta]^\top \in SE(2)$, where x, y represent the linear positions along the x–y axes and θ the orientation.*

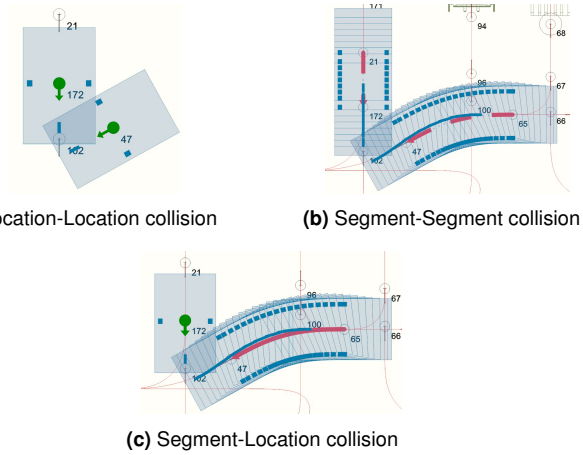
Definition 7. Segment. *A segment r is a continuous curve connecting two locations.*

Definition 8. Roadmap. *A roadmap consists of a set of N^V locations p_u, with $u \in \{1, \dots, N^V\}$, and a set of N^E segments r_h connecting them, with $h \in \{1, \dots, N^E\}$.*

We then provide the definitions associated with the AGVs themselves, characterizing their geometric footprint and the interactions that may give rise to collisions.

Definition 9. Footprint. *The footprint ϕ of an AGV is a convex polygon representing the physical boundary of the vehicle in the workspace. For any pose $q = (x, y, \theta)$, the projection of the footprint is obtained by placing the polygon at (x, y) and rotating it by θ, which defines the region of space occupied by the vehicle at that pose.*

Definition 10. Collision. *Two elements of the roadmap are considered to be in collision if they cannot be occupied by two AGVs simultaneously due to overlap of their footprints, either at locations or along segments. The following cases can be identified (see Fig. 2):*



(a) Location-Location collision

(b) Segment-Segment collision

(c) Segment-Location collision

Figure 2. The figure depicts the three collision cases on the roadmap. Locations are marked by green circles, with their centers corresponding to the x, y coordinates. Each location features a green arrow indicating the robot's orientation θ. Segments are depicted as red arrow curves. Light blue rectangles represent the vehicle's footprints, highlighting their spatial occupancy at both the locations and along the segments.

- *Location-Location: Two locations are in collision if the footprints of two AGVs, occupying them simultaneously, overlap;*
- *Segment-Segment: Two segments are in collision if the footprints of two AGVs simultaneously traversing them overlap at least at one point along their respective segments;*
- *Segment-Location: A segment and a location are in collision if the footprint of an AGV traversing the segment overlaps at least at one point along the segment with the footprint of another AGV occupying the location.*

3.3 Assumptions

The environment of the industrial plant must satisfy the following conditions:

Assumption 1. Predefined Roadmap. *The roadmap is given a priori.*

Assumption 2. Battery Charger. *The number of battery chargers N^B must be at least equal to the number of AGVs N^A, i.e., $N^B \geq N^A$. Battery chargers are positioned in locations such that, when occupied by AGVs, they are not in collision with any segments and locations traversed by other AGVs.*

Regarding the AGVs, the following simplifying assumptions are taken into account:

Assumption 3. Movement and Waiting Constraints. *When operating on the roadmap, the AGVs can only wait at locations or must move along segments.*

Assumption 4. Task Lists. *The Task Lists, i.e., the lists of tasks to be executed for each AGV, are provided by the Task Manager.*

Assumption 5. Fixed Path. *The AGVs follow a fixed path, calculated during task assignment, to perform a specific task.*

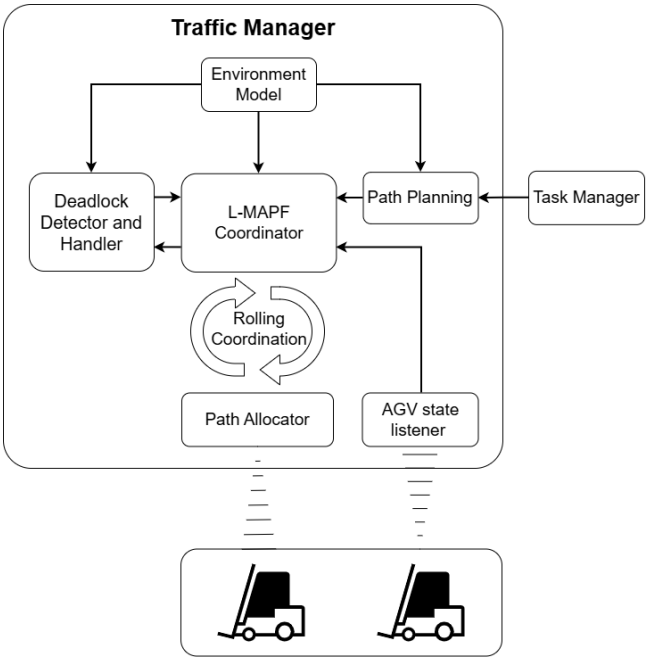


Figure 3. Overview diagram of the proposed traffic management system.

4 Traffic Management Architecture

The proposed strategy adopts a centralized structure to address the traffic management of AGV fleets. The centralized design is selected for its ability to provide a global perspective, enabling the coordination system to maintain a complete and current understanding of the entire framework. This holistic overview facilitates the implementation of optimization algorithms for coordination and deadlock resolution, and ensures consistency in decision-making, which is particularly beneficial in environments characterized by high-traffic density or frequent task alterations (Zhang et al. 2022; Berndt et al. 2021).

Figure 3 illustrates the overall workflow of the proposed strategy. A central control unit, referred to as the Traffic Manager (TM), periodically receives updated information on the state of each AGV, including localization data and status indicators. The TM also acquires the Task Lists provided by the Task Manager, which specify the tasks currently assigned to each vehicle, such as picking up or dropping off goods, navigating toward designated locations, or reaching a charging station when the battery level is low.

The TM first constructs and stores the environment model (see Section 5) and computes the fixed paths required for new task assignments (see Section 6). The L-MAPF coordinator (see Section 7) then periodically computes instances consisting of collision-free trajectories that ensure coherent multi-agent coordination while adapting to task updates and uncertainties. Periodic replanning provides robustness and responsiveness that one-shot planning approaches cannot guarantee.

Since AGVs operate in continuous time and their actual motion may diverge from nominal timings, the trajectories produced by the coordinator cannot be executed directly. The path allocator (see Section 8) therefore serves as the execution layer of the architecture: it enforces mutually exclusive allocations of roadmap elements and issues

safe, real-time movement permissions consistent with the coordinated plan.

Blocking situations may nonetheless occur in dense industrial layouts. Thus, the TM incorporates a dedicated deadlock detection and resolution module, which continuously monitors AGV interactions and replans the paths of the involved vehicles when required (see Section 9).

5 Environment Model

We introduce a hierarchical structure to model the environment composed of two distinct layers: the roadmap layer and the topological layer. The roadmap layer is the representation of the roadmap and is employed for calculating the paths of the AGVs, defining their movements and velocities as they operate in the environment, and modeling their spatial occupancy in order to avoid collisions.

Remark 1. Heterogeneous AGVs. *The fleet operating on the roadmap layer consists of N^A heterogeneous AGVs, each classified within a specific AGV class. Let $\mathcal{C}$ be the set containing all the N^C AGV classes that operate in the environment. A generic AGV class $C_k \in \mathcal{C}$ with $k \in \{1, \dots, N^C\}$ refers to the categorization of AGVs based on the kinematic constraints expressed by:*

$$f_k(q, \dot{q}) = 0 \tag{1}$$

and the convex polygonal footprint ϕ_k, which is applied to the pose that each AGV assumes in the environment.

The topological layer divides the environment into distinct partitions defined as sectors, including temporary storage zones, pick-up and drop-off areas, open spaces, parking zones, and corridors. This segmentation enables the use of customized strategies for AGV coordination based on the specific type of sector being addressed. Particular attention in the present study is given to the corridors, as they play a crucial role in the innovative strategies integrated into the proposed traffic management system.

A visualization of the environment model is depicted in Fig. 4, where the industrial plant is partitioned into various rectangular sectors of the topological layer, each containing segments and locations of the roadmap layer.

5.1 Roadmap Layer

The predefined roadmap is modeled as a directed weighted graph. Specifically, the roadmap layer is a graph $\mathcal{G}^R$ composed of the vertex set $\mathcal{V}^R(\mathcal{G}^R)$ representing the set of specific locations in the environment, and of the edge set $\mathcal{E}^R(\mathcal{G}^R)$, which represents the set of connections between neighboring connected locations, i.e., the roadmap segments.

Remark 2. Heterogeneous Segments. *Roadmap segments are heterogeneous in both shape and length.*

Let p^m, p^n be two locations in the environment, represented by vertices $v^m, v^n \in \mathcal{V}^R(\mathcal{G}^R)$. Then, the edge $e^{m,n} = (v^m, v^n)$ exists in $\mathcal{E}^R(\mathcal{G}^R)$, if a segment exists in the roadmap that directly connects p^m and p^n. For each edge in $\mathcal{E}^R(\mathcal{G}^R)$, an edge weight $\omega^{m,n}$ is set to the average time required to traverse the road segment from p^m to p^n. Hence, the roadmap layer represents N^V locations p_u, with

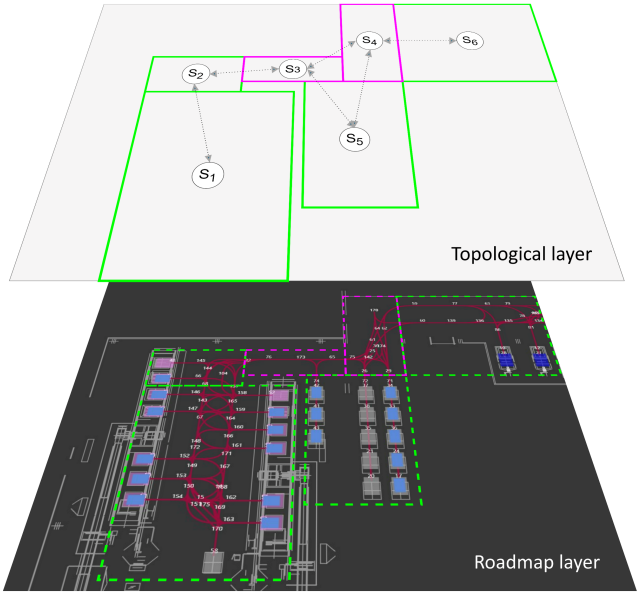


Figure 4. Environment model with roadmap layer and topological layer. The former is composed of segments depicted by red lines, connecting white numbered locations. The latter is characterized by green and magenta sectors, each labeled as S_i with $i \in \{1, \dots, 6\}$. Specifically, the magenta sectors delineate corridors within the environment.

$u \in \{1, \dots, N^V\}$, and N^E roadmap segments r_h with $h \in \{1, \dots, N^E\}$.

Each vertex v^m and edge $e^{m,n}$ of the roadmap layer is designated to a specific AGV class, allowing only vehicles belonging to that class to navigate through them. In particular, each AGV class $C_k \in \mathcal{C}$ with $k \in \{1, \dots, N^C\}$ is associated with a strongly connected subgraph $\mathcal{G}^{C_k}$ of $\mathcal{G}^R$, with the vertex set $\mathcal{V}(\mathcal{G}^{C_k}) \subseteq \mathcal{V}(\mathcal{G}^R)$ and the edge set $\mathcal{E}(\mathcal{G}^{C_k}) \subseteq \mathcal{E}(\mathcal{G}^R)$ representing the vertices and the edges assigned to C_k. Consequently, the graph $\mathcal{G}^R$ is divided in N^C disjoint subgraphs $\mathcal{G}^{C_k}$.

The aforementioned assignment enables edges and vertices to be modeled with kinematic constraints and spatial occupancy specific to each AGV class. Hence, the operability of heterogeneous vehicles in the roadmap layer can be ensured by taking into account the kinematic constraints in (1) and the footprint ϕ_k of each AGV class C_k.

To guarantee compatibility with the AGVs' kinematics and to minimize mechanical wear, the segments represented by each edge set $\mathcal{E}(\mathcal{G}^{C_k})$ must possess adequate smoothness and maintain G^1 continuity (Peters 2002). The latter ensures that adjacent segments share a common tangent direction at their junctions, preventing abrupt changes in curvature or vehicle orientation. Each segment is defined as a NURBS curve whose x and y coordinates satisfy the geometric and curvature constraints in (1) (Liu et al. 2023), with tangent vectors at endpoints matching the vehicle orientation θ at the corresponding locations. As a result, transitions between consecutive segments remain smooth in both position and orientation, enabling kinematically feasible motion across the roadmap, with θ varying continuously along each curve according to the tangent direction of the parametric functions.

Algorithm 1: Collision Sets Calculator

Input: $\mathcal{G}^R, \mathcal{C}$
Output: $\{\mathcal{Y}^m \mid g^m \in \mathcal{V}^R(\mathcal{G}^R) \cup \mathcal{E}^R(\mathcal{G}^R)\}$
1 **foreach** $g^m \in \mathcal{V}^R(\mathcal{G}^R) \cup \mathcal{E}^R(\mathcal{G}^R)$ **do**
2 $\mathcal{Y}^m \leftarrow \{\}$
3 **foreach** $g^n \in \mathcal{V}^R(\mathcal{G}^R) \cup \mathcal{E}^R(\mathcal{G}^R)$ **do**
4 $\phi^m \leftarrow GetFootprint(g^m, \mathcal{C})$
5 $\phi^n \leftarrow GetFootprint(g^n, \mathcal{C})$
6 **if Collision Detection**$(g^m, g^n, \phi^m, \phi^n)$ **then**
7 $\mathcal{Y}^m \leftarrow \mathcal{Y}^m \cup \{g^n\}$
8 **return** $\{\mathcal{Y}^m \mid g^m \in \mathcal{V}^R(\mathcal{G}^R) \cup \mathcal{E}^R(\mathcal{G}^R)\}$

Remark 3. Rotation Segments. *Under Assumption 1, and assuming that C_k classifies vehicles capable of rotating in place, $\mathcal{E}(\mathcal{G}^{C_k})$ can include edges representing "rotation segments". Such segments model stationary rotation maneuvers where the vehicle reorients itself, optimizing its navigation efficiency in constrained spaces. Mathematically, a rotation segment can be defined as a function where the orientation θ varies continuously between an initial angle θ^I and a final angle θ^F, while the linear positions x, y remain constant, such that:*

$$\theta(s) = \theta^I + s(\theta^F - \theta^I), \quad s \in [0, 1] \tag{2}$$

where s is a normalized scalar parameter ranging from 0 to 1, with $s = 0$ corresponding to the initial location p^I, and $s = 1$, corresponding to the final location p^F.

In addition, to ensure operational safety of multiple AGVs on the roadmap layer, it is necessary to account for potential collisions between heterogeneous vehicles as they navigate through shared spaces.

Conventional cell-based collision models (Ziegler and Stiller 2010) become inefficient in the narrow and irregular industrial layouts considered in this work. Achieving sufficient spatial resolution would require a large number of occupancy cells per roadmap element, causing collision checks to scale as $O(N^{OC})$ set-overlap operations, where N^{OC} is the number of occupancy cells associated with the roadmap element being tested. Online bounding-box intersection tests (Zhou and Suri 2000) offer higher geometric precision but still incur non-negligible computational cost when performed repeatedly at runtime, especially for large heterogeneous AGVs moving along curved NURBS segments in confined spaces. To obtain both geometric accuracy and real-time performance, we adopt our proposed collision-set representation.

Accordingly, each generic element $g^m \in \mathcal{V}^R(\mathcal{G}^R) \cup \mathcal{E}^R(\mathcal{G}^R)$ of the roadmap layer is associated with its collision set $\mathcal{D}^m$. Specifically, $\mathcal{D}^m$ contains each generic element $g^n \in \mathcal{V}^R(\mathcal{G}^R) \cup \mathcal{E}^R(\mathcal{G}^R)$, where $g^n \neq g^m$, such that g^m is in collision with g^n, according to Definition 10. In practical terms, the collision set $\mathcal{D}^m$ of g^m contains all, and only, the elements that are in collision with g^m.

Collision sets are evaluated offline, producing compact precomputed sets that allow constant-time $O(1)$ lookup during real-time operation. This approach increases geometric accuracy in confined spaces while reducing computational overhead under high traffic conditions. Specifically, the collision sets are computed using Algorithm 1 - Collision Sets Calculator, which systematically detects collisions between all elements of $\mathcal{G}^R$.

Algorithm 2: Collision Detection

Input: g^m, g^n, ϕ^m, ϕ^n
Output: Boolean
1 **if** g^m is vertex **then**
2 $\mathcal{J}^m \leftarrow \{p^m\}$
3 **else**
4 $\mathcal{J}^m \leftarrow SamplePoses(r^m)$
5 **if** g^n is vertex **then**
6 $\mathcal{J}^n \leftarrow \{p^n\}$
7 **else**
8 $\mathcal{J}^n \leftarrow SamplePoses(r^n)$
9 **foreach** $q^b \in \mathcal{J}^m$ **do**
10 **foreach** $q^d \in \mathcal{J}^n$ **do**
11 $\Phi^m \leftarrow TransformFootprint(\phi^m, q^b)$
12 $\Phi^n \leftarrow TransformFootprint(\phi^n, q^d)$
13 **if** SAT(Φ^m, Φ^n) **then**
14 **return** true
15 **return** false

For each element $g^m \in \mathcal{V}^R(\mathcal{G}^R) \cup \mathcal{E}^R(\mathcal{G}^R)$, a candidate set $\mathcal{Y}^m$ is initialized as an empty set (Algorithm 1, line 2). Algorithm 1 then iterates over each generic element $g^n \in \mathcal{V}^R(\mathcal{G}^R) \cup \mathcal{E}^R(\mathcal{G}^R)$, checking for collisions between g^m and g^n. The process involves: (i) retrieving the footprints ϕ^m and ϕ^n associated with g^m and g^n from the set of AGV classes $\mathcal{C}$ (Algorithm 1, lines 4-5); (ii) applying Algorithm 2 - Collision Detection to determine if g^m and g^n are in collision, according to Definition 10 (Algorithm 1, line 6); (iii) adding g^n to $\mathcal{Y}^m$ if a collision is detected (Algorithm 1, line 7).

Collision detection is performed by Algorithm 2, which takes as input g^m and g^n, along with their respective footprints ϕ^m and ϕ^n. Following the collision cases described in Definition 10, Algorithm 2 first identifies the sets of poses $\mathcal{J}^m$ and $\mathcal{J}^n$ based on whether g^m and g^n represent edges or vertices in $\mathcal{G}^R$. If g^m is an edge, $\mathcal{J}^m$ consists of sampled poses along the represented segment r^m. If g^m is a vertex, $\mathcal{J}^m$ includes the single pose q^m at the location p^m. The same process applies to g^n. For each couple (q^b, q^d), with $q^b \in \mathcal{J}^m$ and $q^d \in \mathcal{J}^n$, Algorithm 2 checks for overlaps between the projections Φ^m, Φ^n of ϕ^m and ϕ^n using the Separating Axis Theorem (SAT) (Ericson 2004). SAT states that two convex shapes do not collide if there is an axis along which their projections do not overlap. If an overlap is detected for at least one couple (q^b, q^d), the algorithm considers g^m and g^n to be in spatial collision, and it returns `true`. Conversely, if no overlap is found, the algorithm returns `false` (Algorithm 2, lines 9-15).

In the following theorem, we state and prove that the output of Algorithm 1 allows the computation of the collision set $\mathcal{D}^m \; \forall g^m \in \mathcal{V}^R(\mathcal{G}^R) \cup \mathcal{E}^R(\mathcal{G}^R)$.

Theorem 1. *Let Assumptions 1 and 3 hold. Let $g^m \in \mathcal{V}^R(\mathcal{G}^R) \cup \mathcal{E}^R(\mathcal{G}^R)$ be a generic element of the roadmap*

layer, and let $\mathcal{D}^m$ be its collision set containing each generic element $g^n \in \mathcal{V}^R(\mathcal{G}^R) \cup \mathcal{E}^R(\mathcal{G}^R)$, such that g^m is in collision with g^n, according to Definition 10. By applying Algorithm 1, we obtain the set $\mathcal{Y}^m = \mathcal{D}^m \; \forall g^m \in \mathcal{V}^R(\mathcal{G}^R) \cup \mathcal{E}^R(\mathcal{G}^R)$.

Proof. We prove the theorem by contradiction. Assume that Algorithm 1 fails to correctly compute $\mathcal{Y}^m$ for at least one $g^m \in \mathcal{V}^R(\mathcal{G}^R) \cup \mathcal{E}^R(\mathcal{G}^R)$, i.e., $\mathcal{Y}^m \neq \mathcal{D}^m$. Two situations can arise:

- False Negative: Suppose $g^n \notin \mathcal{Y}^m$ even though g^m and g^n are in collision, according to Definition 10. This implies that Algorithm 2 returns `false`, despite an actual overlap of the projections Φ^m, Φ^n in at least a couple of poses (q^b, q^d) with $q^b \in \mathcal{J}^m$ and $q^d \in \mathcal{J}^n$. However, this condition leads to a contradiction as Algorithm 2 is designed to detect such overlaps whenever they arise.
- False Positive: Suppose $g^n \in \mathcal{Y}^m$ even though g^m and g^n are not in collision. This implies that Algorithm 2 returns `true` indicating an overlap of projections Φ^m, Φ^n in at least a couple of poses (q^b, q^d) with $q^b \in \mathcal{J}^m$ and $q^d \in \mathcal{J}^n$, even though no such overlap exists. However, this condition leads to a contradiction because Algorithm 2 guarantees the detection of only actual overlaps.

Thus, Algorithm 1 avoids both false negatives and false positives. Therefore, for every g^m, $g^n \in \mathcal{Y}^m$ if and only if $g^n \in \mathcal{D}^m$, i.e., $\mathcal{Y}^m = \mathcal{D}^m$.

In simple terms, according to Definition 10, $\mathcal{D}^m$ is the true collision set of g^m, containing all elements that intersect the footprint of the vehicle associated with g^m. Conversely, the set $\mathcal{Y}^m$ is the one computed by Algorithm 1. Therefore, the proof shows that the algorithm neither adds nor misses elements, so $\mathcal{Y}^m$ coincides with $\mathcal{D}^m$.

Remark 4. Sampling Accuracy. *Under Assumptions 1 and 3, the sampling process in Algorithm 2 must sample poses on segments finely enough to accurately capture the set $\{\mathcal{D}^m \mid g^m \in \mathcal{V}^R(\mathcal{G}^R) \cup \mathcal{E}^R(\mathcal{G}^R)\}$.*

5.2 Topological Layer

The upper level consists of a topological map, serving as an abstract representation of the environment in which the AGVs operate. The map defines the spatial relationships among different areas of the facility and the paths accessible to the AGVs. Hence, it provides a topological representation of the industrial plant as a set of N^S sectors, e.g., corridors.

Specifically, the set of interconnected sectors defines a directed and connected unweighted graph $\mathcal{G}^{IS}$, referred to as the topological layer. Each vertex $S \in \mathcal{V}^{IS}(\mathcal{G}^{IS})$ represents a sector, which is associated with the subgraph $\mathcal{G}^S$ of $\mathcal{G}^R$, whose vertex set and edge set are given by $\mathcal{V}(\mathcal{G}^S) \subseteq \mathcal{V}(\mathcal{G}^R)$ and $\mathcal{E}(\mathcal{G}^S) \subseteq \mathcal{E}(\mathcal{G}^R)$, respectively. $\mathcal{V}(\mathcal{G}^S)$ contains all the vertices of $\mathcal{G}^R$ representing locations within the sector, while $\mathcal{E}(\mathcal{G}^S)$ contains all the edges of $\mathcal{G}^R$ that connect two vertices in $\mathcal{V}(\mathcal{G}^S)$. The edge set $\mathcal{E}^{IS}(\mathcal{G}^{IS})$ represents the set of connections between neighboring connected sectors. Let $S^b, S^d \in \mathcal{V}^{IS}(\mathcal{G}^{IS})$ be two sectors, and let $v^m \in \mathcal{G}^{S^b}$ and $v^n \in \mathcal{G}^{S^d}$ be two vertices contained in their respective subgraphs. Then, an edge (S^b, S^d) that connects S^b and S^d exists in $\mathcal{E}^{IS}(\mathcal{G}^{IS})$, if an edge exists in $\mathcal{G}^R$ that connects the vertex v^m to the vertex v^n.

Hence, the topological layer is divided in N^S sectors S_s, with $s \in \{1, \dots, N^S\}$.

6 Path Generation and Trajectory Definition

This section introduces the elements required to define the paths and trajectories employed in the proposed traffic management architecture. Since both the spatial path and the associated trajectory depend on the start and goal vertices specified by each task, the discussion begins by describing how tasks are represented within the system. Although the TM does not perform task assignment, it must operate on the tasks supplied by the Task Manager, and their structure must therefore be clearly stated. Task assignment itself falls beyond the scope of this work; interested readers may refer to Sabattini et al. (2015) and Ren et al. (2024) for previous studies on the topic.

Following the specification of the task structure, the section describes the computation of the fixed path derived from each task, in accordance with Assumption 5, and subsequently introduces its trajectory counterpart, which incorporates the temporal evolution of the AGV along the predefined path.

The system consists of N^A heterogeneous AGVs (see Remark 1), each capable of performing tasks or remaining idle at the battery charger. According to Assumption 4, each AGV is assigned a Task List detailing the tasks to be completed, which can be categorized as:

- *Operational tasks*, which involve the execution of activities such as the pick-up and delivery of pallets within the operational environment.
- *Return-to-battery-charger tasks*, which require the AGV to navigate to a designated charging station for battery recharging.

When a generic a-th AGV completes its current task or remains idle at the charger, the TM checks its Task List for pending tasks. If a new task is available, it is removed from the list and assigned to the AGV. Each task is associated with a starting vertex $v^{s,a}$, corresponding to the AGV's current location during task assignment, and a goal vertex $v^{g,a}$, which must be reached to complete the current task. For *operational tasks*, the Task List also includes the next task, with an associated next goal vertex $v^{g',a}$, which remains stored but not yet execute. Note that if the task is a *return-to-battery-charger*, then $v^{g,a} = v^{g',a}$.

The consideration of future tasks is crucial. Prior work on L-MAPF has shown that incorporating information on upcoming tasks improves coordination performance (Li et al. 2021; Grenouilleau et al. 2019a). A similar principle applies in the proposed TM: by accounting for the next task, the TM avoids assuming that each a-th AGV remains indefinitely at its current goal vertex $v^{g,a}$, leading to coordination choices that eventually produce deadlocks.

While this strategy is valid for *return-to-battery-charger tasks* due to Assumption 2, it is unsuitable for *operational*

tasks because of Assumption 5. Such limitation can lead to two distinct deadlock scenarios:

- Let a-th and b-th be two AGVs assigned with a task, such that the goal vertex of the a-th AGV is $v^{g,a}$, and the b-th AGV must traverse edges that conflict with $v^{g,a}$ to complete its task. When the a-th AGV reaches its goal $v^{g,a}$, the TM may incorrectly assume that it will remain stationary indefinitely. Consequently, the TM may coordinate the b-th AGV in a way that causes both AGVs to block each other during the execution of the a-th AGV's subsequent task.

- If both the a-th and b-th AGVs are assigned the same goal vertex, $v^{g,a} = v^{g,b}$, the first AGV to reach $v^{g,a}$ may be blocked by the arrival of the second AGV.

Hence, proactively coordinating AGVs even after they have reached their current goal vertex enables the TM to anticipate and resolve potential space-time collisions arising when the next task is assigned. This approach ensures smoother task transitions and minimizes system downtime, effectively preventing the deadlock scenarios outlined above.

To enhance clarity and facilitate a better understanding of the TM coordination process, it is helpful to classify AGVs based on their current task assignment status.

Remark 5. Classification of AGVs as Active and Free. *AGVs assigned to a task are referred to as active AGVs and belong to the set $\mathcal{A}$, whereas AGVs not assigned to a task are called free AGVs. When a task is assigned to a free AGV, such AGV is added to $\mathcal{A}$. Conversely, when an AGV returns to the battery charger, i.e., when it is no longer assigned any tasks, it is removed from $\mathcal{A}$.*

According to Assumption 5, each a-th active AGV is associated with the fixed path π^a. The path π^a is determined during the task assignment process by extending the main path $\pi^{M,a}$ calculated from $v^{s,a}$ to $v^{g,a}$ of length $L^{M,a}$ with the path $\pi^{N,a}$ from $v^{g,a}$ to $v^{g',a}$ of length $L^{N,a}$. Both paths are calculated using a generic graph search algorithm (Edelkamp and Schrödl 2012) on the subgraph $\mathcal{G}^{C_k}$ of $\mathcal{G}^R$. Specifically, π^a is defined on $\mathcal{G}^{C_k}$ as an ordered sequence of vertices:

$$\begin{aligned} \pi^a = \{&v_1^a, v_2^a, \ldots, v_{L^{M,a}}^a, \ldots, v_{L^a}^a\}, \\ &v_i^a \in \mathcal{V}(\mathcal{G}^{C_k}), \forall i = 1, \ldots, L^a \end{aligned} \tag{3}$$

where $v_1^a = v^{s,a}$, $v_{L^{M,a}}^a = v^{g,a}$, $v_{L^a}^a = v^{g',a}$, and L^a is the total path length. Specifically, L^a is given by:

$$L^a = L^{M,a} + L^{N,a} - 1$$

with -1 accounting for the shared vertex $v^{g,a}$ between $\pi^{M,a}$ and $\pi^{N,a}$, included only once in the path π^a.

Remark 6. Return-to-Battery-Charger Path. *If the assigned task is a return-to-battery-charger task, i.e., $v^{g,a} = v^{g',a}$, then $\pi^{N,a} = \{v^{g,a}\}$, $L^N = 1$, and $\pi^a = \pi^{M,a}$.*

Each pair of consecutive vertices v_i^a and v_{i+1}^a is connected by an edge $e_{i,i+1}^a \in \mathcal{E}(\mathcal{G}^{C_k})$, for $i \in \{1, \ldots, L^a - 1\}$. The path π^a minimizes the total traversal cost:

$$\Omega^a = \sum_{i=1}^{L^a-1} \omega_{i,i+1}^a \tag{4}$$

where $\omega_{i,i+1}^a$ is the cost of traversing the edge $e_{i,i+1}^a$. Edge costs can include dynamic weight adjustments based on complex factors like traffic status (Pratissoli et al. 2023) or traffic predictions (Liu et al. 2021) to optimize flow. However, since this work focuses on coordination strategies for smooth fleet operation in high-density, non-standardized environments rather than path planning methodologies, edge costs are simplified to the average travel times of the represented segments, excluding traffic density distribution and other dynamic factors (Tien and Nguyen 2022).

To prevent collisions and complete its task, according to Assumption 3, each a-th active AGV may either move along an edge $e_{i,i+1}^a$ connecting consecutive vertices $v_i^a, v_{i+1}^a \in \pi^a$, or wait at a vertex v_i^a of its predetermined path π^a. A move action executed along edge $e_{i,i+1}^a$ is represented by the tuple

$$\{v_i^a, v_{i+1}^a, \tau^a, D_{i,i+1}^a\}, \tag{5}$$

where τ^a is the starting timestep and $D_{i,i+1}^a$ is the traversal duration, expressed in discrete timesteps and equal to the edge cost $\omega_{i,i+1}^a$. Conversely, a wait action is represented by

$$\{v_i^a, v_i^a, \tau^a, D_{i,i}^a\}, \tag{6}$$

where $D_{i,i}^a$ corresponds to one timestep. A trajectory γ^a for the a-th AGV is therefore defined as an ordered sequence of $N^{\gamma,a}$ actions

$$\{v_{i(k)}^a, v_{j(k)}^a, \tau_k^a, D_{i(k),j(k)}^a\}, \quad k \in \{1, \ldots, N^{\gamma,a}\}, \tag{7}$$

with $j(k) = i(k)$ (wait) or $j(k) = i(k) + 1$ (move).

7 L-MAPF Coordinator

The L-MAPF coordinator is responsible for generating space-time consistent trajectories for all active AGVs, ensuring collision-free motion while enforcing the fixed paths assigned to each vehicle and respecting the structural constraints of the industrial layout. Unlike one-shot planning strategies, the L-MAPF coordinator operates in a rolling horizon fashion: instances, each consisting of a set of trajectories, are periodically recomputed so that the system can continuously adapt to new task assignments, evolving AGV states, and execution uncertainties.

A rigorous formulation of the coordination problem solved at each instance requires a precise specification of all quantities involved in trajectory generation. Relevant elements include the current target position associated with each AGV, the fixed path along which coordination is performed, the roadmap subgraphs used during planning, and the static and dynamic obstacles that represent environmental and traffic constraints.

Section 7.1 introduces and organizes all components required as inputs to the L-MAPF coordination framework for computing an instance. Section 7.2 then provides the formal formulation of the coordination problem solved at each instance. Section 7.3 concludes the discussion by describing the Anytime Bounded Horizon CBS (ABH-CBS) algorithm, which computes the corresponding set of coordinated trajectories.

Table 2. L-MAPF Coordinator Inputs.

	Input	Description
I	$\delta^{'}$	Base Time Horizon
II	$\delta^{''}$	Horizon Increment
III	t^{σ}	Timeout
IV	$\mathcal{V}^{z} = \{v_z^a \vert a \in \mathcal{A}\}$	Target Vertices
V	$\mathcal{T}^{z} = \{\tau_z^a \vert a \in \mathcal{A}\}$	Target Timesteps
VI	$\mathcal{V}^{g^{'}} = \{v^{g^{'},a} \vert a \in \mathcal{A}\}$	Next Goal Vertices
VII	$\mathcal{K}^{\text{tot}} = \{\mathcal{K}^a \vert a \in \mathcal{A}\}$	Path Subgraphs
VIII	$\mathcal{O}^{s}$	Static Obstacles
IX	$\mathcal{O}^{d} = \{\mathcal{U}^{\setminus a} \vert a \in \mathcal{A}\}$	Dynamic Obstacles
X	$\mathcal{S} = \{\xi^a \vert a \in \mathcal{A}\}$	Extended Corridors

7.1 Coordination Problem Inputs

To define an instance, several inputs must be specified, as summarized in Table 2 and detailed in the sequel.

The coordination process operates within a base horizon of $\delta^{'}$ timesteps (Row I), which may be extended in corridor sectors when required for safe conflict resolution. Outside corridors, the horizon can be incrementally expanded by $\delta^{''}$ timesteps (Row II) until the timeout t^{σ} is reached (Row III).

Each a-th active AGV is associated with a target vertex v_z^a with $z \in \{1, \dots, L^{M,a}\}$, provided by the path allocator (see Section 8). The target vertex represents the discrete location where the newly computed trajectory must start, corresponding to the endpoint of the last edge assigned to the a-th AGV. The set of all target vertices, $\mathcal{V}^z = \{v_z^a \vert a \in \mathcal{A}\}$ (Row IV) is therefore provided as input.

Let $v_l^a \in \pi^a$ denote the currently occupied vertex of the a-th active AGV, with $l \in \{1, \dots, L^{M,a}\}$ and $l \leq z$. We define the allocated trajectory u^a as the ordered sequence of move actions

$$u^a = \left\{ \{v_i^a, v_{i+1}^a, \tau^a, D_{i,i+1}^a\} \;\middle|\; i \in \{l, \dots, z-1\} \right\}, \quad (8)$$

which connects the currently occupied vertex v_l^a to the target vertex v_z^a. The resulting sequence represents the portion of π^a that the AGV must traverse without interruption.

The target timestep τ_z^a, belonging to the set $\mathcal{T}^z$ (Row V), denotes the discrete time at which the AGV is expected to reach v_z^a. The corresponding value is obtained from u^a as:

$$\tau_z^a = \tau_{z-1}^a + D_{z-1,z}^a. \quad (9)$$

Hence, at the target timestep τ_z^a, the L-MAPF coordinator calculates a trajectory γ^a for each a-th active AGV from the current target vertex v_z^a to the next goal vertex $v_{g'}^a$, contained in the set $\mathcal{V}^{g^{'}}$ (Row VI).

Remark 7. Target Timestep with Empty Allocated Queue. *If the allocated queue w^a of the a-th active AGV is empty, i.e., $u^a = \{\}$, then the target timestep of the a-th active AGV is $\tau_z^a = 0$.*

According to Assumption 5, each active a-th AGV must be coordinated along its fixed path π^a. Thus, coordination is performed on the subgraph $\mathcal{K}^a$, defined as $\mathcal{K}^a = (\mathcal{V}^a, \mathcal{E}^a)$ of $\mathcal{G}^{C_k} = (\mathcal{V}^{C_k}, \mathcal{E}^{C_k})$, where $\mathcal{V}^a \subseteq \mathcal{V}^{C_k}$ and $\mathcal{E}^a \subseteq \mathcal{E}^{C_k}$. $\mathcal{K}^a$ contains the vertices $\{v_1^a, v_2^a, \dots, v_L^a\}$ and edges $\{e_{1,2}^a, e_{2,3}^a, \dots, e_{L^a-1,L^a}^a\}$. Hence, we define the set $\mathcal{K}^{\text{tot}} = \{\mathcal{K}^a \vert a \in \mathcal{A}\}$ (Row VII) containing all path subgraphs of active AGVs.

Remark 8. Goal Vertex Traversal Constraint. *Each trajectory γ^a is constrained to traverse the goal vertex $v^{g,a}$. This is automatically ensured by computing γ^a on the subgraph $\mathcal{K}^a$, which contains only vertices and edges associated with the fixed path π^a.*

To calculate collision-free trajectories, the L-MAPF coordinator must account for obstacles, which can be categorized as static or dynamic. On the one hand, static obstacles consist of unintended obstructions, e.g., shelving units and pallets, that block specific edges of the roadmap layer, potentially causing collisions with active AGVs. To prevent the computation of colliding trajectories, each static obstacle o_k, where $k \in \{1, \dots, N^O\}$ with N^O the total number of obstructions, is associated with a subset of obstructed edges $\mathcal{O}_k \subseteq \mathcal{E}^R$. Consequently, the static obstacles are represented by the set $\mathcal{O}^s \subseteq \mathcal{E}^R$ (Row VIII), defined as follows:

$$\mathcal{O}^s = \bigcup_{k=1}^{N^O} \mathcal{O}_k. \quad (10)$$

On the other hand, dynamic obstacles represent the time-varying occupation of roadmap edges generated by the portions of the path allocated for other AGVs to traverse. Since the allocated trajectory u^a represents the segment of π^a that the a-th AGV must traverse without interruption, it must be treated as a dynamic obstacle when planning the trajectories of the others AGVs. Consequently, the L-MAPF coordinator takes as input the set $\mathcal{O}^d = \{\mathcal{U}^{\setminus a} \vert a \in \mathcal{A}\}$ (Row IX), which contains the dynamic obstacles for each a-th active AGV, defined as

$$\mathcal{U}^{\setminus a} = \mathcal{U} \setminus u^a. \quad (11)$$

To ensure effective coordination, the trajectory γ^a for each a-th AGV must also account for potential challenges posed by shared resources, such as narrow bidirectional corridors. Each corridor, defined in the topological layer as a sector $S \in \mathcal{V}^{IS}(\mathcal{G}^{IS})$ (see Section 5.2), requires complete conflict resolution to prevent deadlocks. While our system operates within a bounded horizon, this may not be adequate for long, narrow bidirectional corridors. By applying our Extended Horizon in Corridor (EHC) strategy proposed in Bonetti et al. (2024b), we can effectively manage AGV coordination within any corridor S.

However, since the segments of the roadmap layer vary in length and shape (see Remark 2), deadlocks could potentially occur outside the boundaries of corridors (see Fig. 5). Thus, following the Corridor eXtension (CX) procedure described in our previous work (Bonetti et al. 2024b), we define the set $\mathcal{S} = \{\xi^a \vert a \in \mathcal{A}\}$ (Row X), which includes the extended corridor $\xi^a \supseteq \pi^a$ for each a-th active AGV. In particular, ξ^a includes all the vertices of π^a, both within and adjacent to the corridor sectors where the EHC strategy is applied.

Each time a new path is calculated during task assignment or an active AGV returns to a battery charger (see Remark 5), $\mathcal{S}$ is updated by comparing the paths of all active AGVs as outlined in Algorithm 3 - Extended Corridors Update.

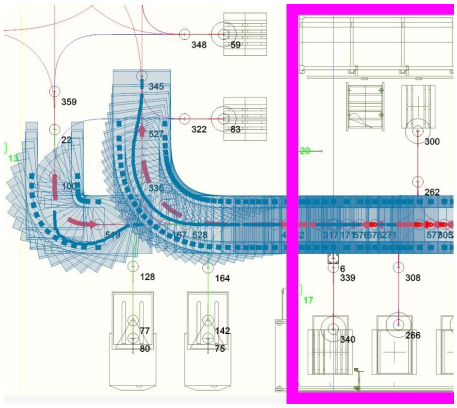


Figure 5. Example of a corridor extension with the corridor sector delimited by magenta borders, where one AGV intends to enter while another needs to exit.

Algorithm 3: Extended Corridors Update

Input: $\mathcal{P}, \mathcal{G}^{IS}$
Output: $\mathcal{S}$
1 $\mathcal{S} \leftarrow \{\xi^a \leftarrow \{\} \mid a \in \mathcal{A}\}$
2 **foreach** $\pi^a, \pi^b \in \mathcal{P}, \pi^a \neq \pi^b$ **do**
3 **foreach** $S \in \mathcal{V}^{IS}(\mathcal{G}^{IS})$ **do**
4 **if** S is Corridor **then**
5 $\mathcal{G}^S \leftarrow GetCorridorSubgraph(S)$
6 $\xi^a \leftarrow \xi^a \cup ExtendedCorridor(\pi^a, \pi^b, \mathcal{G}^S)$
7 $\xi^b \leftarrow \xi^b \cup ExtendedCorridor(\pi^b, \pi^a, \mathcal{G}^S)$
8 **return** $\mathcal{S}$

Algorithm 3 takes as input the topological layer $\mathcal{G}^S$ and the set $\mathcal{P} = \{\pi^a \mid a \in \mathcal{A}\}$, which includes the path of each a-th active AGV, and returns the updated set $\mathcal{S}$. The process begins by initializing $\mathcal{S}$ with an empty extended corridor for each a-th AGV (Algorithm 3, line 1). Subsequently, the algorithm compares the paths of each pair of AGVs, π^a and π^b, applying the CX strategy to each corridor sector to fill the respective extended corridors ξ^a and ξ^b (Algorithm 3, lines 2-7). Through an exhaustive analysis of all possible path combinations, Algorithm 3 calculates the updated set $\mathcal{S}$, which is provided as input for the L-MAPF coordinator (Row X).

7.2 Coordination Problem Definition

Having defined all required inputs in Section 7.1, we now formally state the coordination problem solved by the L-MAPF coordinator for each instance.

Problem 1. Coordination Problem. *Given:*

- *the set of active AGVs $\mathcal{A}$;*
- *for each $a \in \mathcal{A}$, the fixed path π^a;*
- *for each $a \in \mathcal{A}$, the target vertex v_z^a and the target timestep τ_z^a with $z \in \{1, \ldots, L^{M,a}\}$;*
- *for each $a \in \mathcal{A}$, the goal vertex $v^{g',a}$;*
- *the set of static obstacles $\mathcal{O}^s$;*
- *the set of dynamic obstacles $\mathcal{O}^d$;*

the coordination problem consists in computing a set of collision-free trajectories

$$\mathcal{H}^\star = \{\gamma^{a,\star} \mid a \in \mathcal{A}\},$$

such that:

- *each $\gamma^{a,\star}$ begins at v_z^a at time τ_z^a;*
- *each $\gamma^{a,\star}$ reaches $v^{g',a}$ in finite time, if a solution exists;*
- *each $\gamma^{a,\star}$ adheres to the corresponding fixed path π^a;*
- *all trajectories are mutually conflict-free;*
- *all trajectories avoid static and dynamic obstacles;*

Among all feasible sets of trajectories, the L-MAPF coordinator tries to minimize a global performance metric J, which corresponds to the Sum of Costs (SoC). The chosen metric captures the overall travel effort of the fleet and promotes solutions that reduce unnecessary delays and congestion. In particular, SoC is computed as the sum of the travel times of the individual trajectories, defined as the completion time of their last action:

$$J = \sum_{\gamma^{a,\star} \in \mathcal{H}^\star} \left(\tau_k^a + D^a_{i(k),\,j(k)}\right), \qquad k = N^{\gamma,a}. \tag{12}$$

Finding an optimal solution to Problem 1 has been shown to be NP-hard (Yu and LaValle 2013). The problem, in fact, admits a reduction to a MAPF instance defined on a suitably constructed reduced graph, where the combinatorial growth generated by temporal interactions among multiple agents remains fully preserved. The resulting complexity renders optimality infeasible in realistic industrial settings, where AGVs operate in dense and constrained environments and must react in real-time to traffic variations, task updates, and execution uncertainties.

The adopted strategy therefore focuses on computing locally optimal solutions, preserving coordination quality while keeping the computational burden tractable. To achieve such a balance, the L-MAPF coordinator employs the ABH-CBS algorithm, which builds upon the RHCR framework (Li et al. 2021). Every η timesteps, the algorithm periodically solves bounded-horizon instances and progressively extends the planning window through an anytime mechanism.

In operational terms, at each instance, ABH-CBS first solves the coordination problem restricted to the base time horizon δ'. The algorithm then applies selective horizon extensions within the extended corridors $\mathcal{S}$, ensuring that potential conflicts in the considered regions are fully resolved. Once a feasible solution is identified, the planning window is enlarged by the horizon increment δ'', and the bounded-horizon instance is refined, repeatedly expanding the horizon until the global timeout t^σ is reached. As the horizon grows, the sequence of locally optimal solutions may eventually converge to an optimal and complete solution when the window becomes sufficiently large to cover the full arrival of all AGVs at their respective goals; under such conditions, ABH-CBS solution coincides with the one of standard CBS.

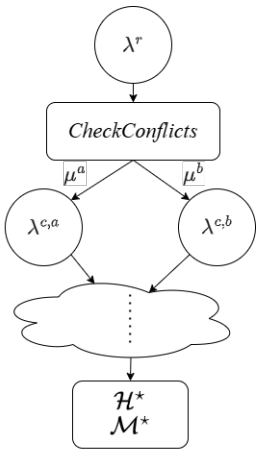


Figure 6. Constraint Tree expansion of the ABH-CBS algorithm, detailing the process leading to the computation of the outputs $\mathcal{H}^\star$ and $\mathcal{M}^\star$.

7.3 Anytime Bounded Horizon CBS

The proposed ABH-CBS utilizes a hybrid approach consisting of two levels (Sharon et al. 2015): a low-level perspective that deals with individual AGVs and a high-level perspective that views the system holistically. On the one hand, the low-level is responsible for planning trajectories for each a-th active AGV that meet the constraints assigned by the high-level and that avoid static and dynamic obstacles. On the other hand, the high-level involves searching for space-time collision among AGV trajectories, defining constraints, and allocating them to AGVs for the low-level planning.

The low-level planner is based on the space-time A* algorithm (Silver 2005), modified to operate effectively on a roadmap consisting of heterogeneous segments and to align with the assumptions and obstacle representation defined in this work. Specifically, the low-level planner (as detailed in the Appendix A) calculates a trajectory γ^a for the a-th active AGV from v_z^a to $v^{g',a}$ on the subgraph $\mathcal{K}^a$, while adhering to constraints imposed by the high-level planner and avoiding both dynamic and static obstacles.

The high-level planner receives the inputs reported in Table 2 (see Section 7.2) and utilizes a best-first approach to explore a binary tree named Constraint Tree (CT) (Sharon et al. 2015), as shown in Fig. 6. In contrast to our previous work (Bonetti et al. 2024b), the planner has been modified to incorporate anytime functionality. This enhancement enables the computation of conflict-free trajectories within a bounded horizon, which are progressively refined toward optimality as computation time permits.

Each generic node within the CT comprises four key elements:

- *Node Constraint*: this consists of a set $\mathcal{M}$ of constraints μ_k, with $k \in \{1, 2, \dots\}$, accumulated throughout the expansion of the CT. Each element μ_k restricts the movement of the generic a-th active AGV between two neighbour vertices, i.e., edge $e^a_{i,i+1}$, or forbids wait action on vertex v^a_i, during a specific timestep interval in the low-level planner. In particular, μ_k is expressed as a tuple $(a, v^a_i, v^a_j, [\tau^I, ..., \tau^F], b)_k$, with $j = i \vee i+1$ for the a-th AGV. Here, $[\tau^I, \dots, \tau^F]$ represents the constrained timestep interval, spanning from the initial timestep τ^I to the final timestep τ^F, imposed by the b-th AGV.

- *Node Solution*: this includes a set of AGVs' timestep-minimal trajectories $\mathcal{H} = \{\gamma^a | a \in \mathcal{A}\}$, calculated by the low level planner and satisfying the constraints defined in $\mathcal{M}$.

- *Node Cost*: this represents the total cost of the node, calculated as $\sum_{\gamma^a \in \mathcal{H}} |\gamma^a|$, where $|\gamma^a|$ denotes the cost of each trajectory in $\mathcal{H}$. Since $|\gamma^a|$ corresponds to the trajectory travel time, the node cost implements the SoC, ensuring consistency with the coordination metric J in Problem 1.

- *Node Horizon*: this consists of the set $\mathcal{N}$, which includes the distinct time horizon value δ^a for each a-th AGV. In particular, δ^a is set as the current value of δ in the non-corridor zones of the environment. If the AGV is inside its extended corridor ξ_a at the timestep δ^a along the timestep-minimal trajectory $\gamma^a \in \mathcal{H}$, then δ^a is extended until the a-th AGV reaches the first vertex $v^{\text{out},a} \notin \xi_a$ using the EHC strategy (Bonetti et al. 2024b).

Algorithm 4: High-Level Planner

Input: $\delta^{'}, \delta^{''}, t^\sigma, \mathcal{V}^z, \mathcal{T}^z, \mathcal{V}^{g'}, \mathcal{K}^{\text{tot}}, \mathcal{O}^s, \mathcal{O}^d, \mathcal{S}$
Output: $\mathcal{H}^\star, \mathcal{M}^\star$

$t^\iota \leftarrow t$
$\delta \leftarrow \delta^{'}$
$\mathcal{H}^\star \leftarrow \emptyset$
$\mathcal{M}^\star \leftarrow \emptyset$
$\lambda^r \leftarrow GenerateRoot(\mathcal{V}^z, \mathcal{T}^z, \mathcal{V}^{g'}, \mathcal{O}^s, \mathcal{O}^d, \mathcal{K}^{\text{tot}}, \mathcal{S}, \delta)$
OPEN_H $\leftarrow \{\lambda^r\}$
while OPEN_H $\neq \emptyset$ **and** $t - t^\iota < t_s$ **do**
  $\lambda^p \leftarrow$ lowest *Cost* node from OPEN_H
  OPEN_H $\leftarrow$ OPEN_H $\setminus \lambda^p$
  foreach $\delta^a \in \mathcal{N}(\lambda^p)$ **do**
    if $\delta^a < \delta$ **then**
      $\delta^a \leftarrow \delta$
      $\delta^a \leftarrow EHC(\delta^a, \mathcal{S}, \mathcal{H}(\lambda^p))$
  $[\mu^a, \mu^b] \leftarrow CheckConflicts(\mathcal{H}(\lambda^p), \mathcal{N}(\lambda^p), \mathcal{K}^{\text{tot}})$
  if $\mu^a \neq \varnothing$ ***and*** $\mu^b \neq \varnothing$ **then**
    $\lambda^{c,a} \leftarrow GenerateChild(\lambda^p, \mathcal{V}^z, \mathcal{T}^z, \mathcal{V}^{g'}, \mathcal{O}^s, \mathcal{O}^d, \mathcal{K}^{\text{tot}}, \mathcal{S}, \delta, \mu^a)$
    $\lambda^{c,b} \leftarrow GenerateChild(\lambda^p, \mathcal{V}^z, \mathcal{T}^z, \mathcal{V}^{g'}, \mathcal{O}^s, \mathcal{O}^d, \mathcal{K}^{\text{tot}}, \mathcal{S}, \delta, \mu^b)$
    if $\lambda^{c,a}$ is valid **then**
      OPEN_H $\leftarrow$ OPEN_H $\cup \{\lambda^{c,a}\}$
    if $\lambda^{c,b}$ is valid **then**
      OPEN_H $\leftarrow$ OPEN_H $\cup \{\lambda^{c,b}\}$
  else
    $\mathcal{H}^\star \leftarrow \mathcal{H}(\lambda^p)$
    $\mathcal{M}^\star \leftarrow \mathcal{M}(\lambda^p)$
    $\delta \leftarrow \delta + \delta^{''}$
    OPEN_H $\leftarrow$ OPEN_H $\cup \{\lambda^p\}$
return $\mathcal{H}^\star, \mathcal{M}^\star$

Algorithm 4 - High-Level Planner begins with an initialization phase, which involves recording the current time t in t^ι, setting the time horizon δ with the base

time horizon $\delta^{'}$, and initializing the algorithm outputs $\mathcal{H}^{\star}$ and $\mathcal{M}^{\star}$ as empty sets (Algorithm 4, lines 1-4). Subsequently, Algorithm 4 computes the root node λ^{r} using the *GenerateRoot* function and inserts it in a list, in the sequel referred to as the OPEN_H list (Algorithm 4, lines 5-6). λ^{r} is characterized by an initial solution $\mathcal{H}$ planned by the low-level with an empty set of constraints $\mathcal{M} = \emptyset$. At this step, the root node *Cost* and *Horizon* fields are also computed.

As long as the OPEN_H list is not empty and the current computation time $t - t^{\iota}$ does not exceed the calculation timeout t^{σ}, the node λ^{p} with the lowest cost, referred to as the parent node, is selected and removed from the OPEN_H list (Algorithm 4, lines 8-9). Then, Algorithm 4 performs the horizon update phase on λ^{p}, which consists of checking whether each $\delta^{a} \in \mathcal{N}$ of λ^{p} is less than the current time horizon δ. If this condition holds, δ^{a} is set equal to δ and the EHC strategy is applied to $\gamma^{a} \in \mathcal{H}$ of the corresponding a-th active AGV (Algorithm 4, lines 10-13).

Algorithm 4 proceeds with conflict detection among all pairs of trajectories in $\mathcal{H}$ of λ^{p} using the *CheckConflicts* function (Algorithm 4, line 14). Let $a, b \in \mathcal{A}$ be two active AGVs and let γ^{a} and γ^{b} be the respective trajectories included in $\mathcal{H}$. The function checks for space-time collisions between each pair of actions $\{v_{i}^{a}, v_{j}^{a}, \tau^{a}, D_{i,j}^{a}\} \in \gamma^{a}, \{v_{m}^{b}, v_{n}^{b}, \tau^{b}, D_{m,n}^{b}\} \in \gamma^{b}$ using Algorithm 9 - Check Space-Time Collision (see Appendix A). The search is conducted within a bounded horizon defined by the minimum value between δ^{a} and δ^{b}, as described in (Bonetti et al. 2024b).

On the one hand, if conflicts exist, *CheckConflicts* selects the first identified conflict between the a-th and b-th AGVs and generates the constraints $[\mu^{a}, \mu^{b}]$ to resolve it in the child nodes. In particular, $[\mu^{a}, \mu^{b}]$ are defined as:

$$\begin{aligned}&[(a, v_{i}^{a}, v_{j}^{a}, [\tau^{b}, ..., \tau^{b} + D_{m,n}^{b}], b),\\ &(b, v_{m}^{b}, v_{n}^{b}, [\tau^{a}, ..., \tau^{a} + D_{i,j}^{a}], a)].\end{aligned} \tag{13}$$

Specifically, the action from vertex v_{i}^{a} to v_{j}^{a} in μ^{a} is constrained for the time interval $[\tau^{b}, ..., \tau^{b} + D_{m,n}^{b}]$ in which $\{v_{m}^{b}, v_{n}^{b}, \tau^{b}, D_{m,n}^{b}\} \in \gamma^{b}$ conflicts with it and vice versa. Then, Algorithm 4 generates the child nodes $\lambda^{c,a}$ and $\lambda^{c,b}$ using the *GenerateChild* function (Algorithm 4, lines 16-17). Specifically, *GenerateChild* incorporates the constraints μ^{a} and μ^{b} into the respective constraint sets of $\lambda^{c,a}$ and $\lambda^{c,b}$ and invokes the low-level planner to replan the trajectories γ^{a} and γ^{b} for the a-th and b-th AGVs involved in the selected conflict. In addition, the function updates the $Cost$ and $Horizon$ fields of the child nodes accordingly. Hence, if the new trajectories γ^{a}, γ^{b} are successfully computed by the low-level planner, the child nodes $\lambda^{c,a}$ and $\lambda^{c,b}$ are deemed valid, and Algorithm 4 adds them to the OPEN_H list (Algorithm 4, lines 18-21).

On the other hand, if there are no conflicts, $\mathcal{H}$ of λ^{p} is stored in $\mathcal{H}^{\star}$, the constraints set $\mathcal{M}$ is stored in $\mathcal{M}^{\star}$, and the time horizon δ is incremented by $\delta^{''}$ (Algorithm 4, lines 23-25). Then, the parent node λ^{p} is re-inserted in the OPEN_H list (Algorithm 4, line 26).

The high-level ABH-CBS repeats this process until the current computation time exceeds the calculation timeout t^{σ} or all possibilities are exhausted, i.e., the OPEN_H list is empty. It then returns as output the last assignment of $\mathcal{H}^{\star}$ corresponding to the lowest-cost solution found within the explored planning horizon that minimize J, together with the associated constraint set $\mathcal{M}^{\star}$.

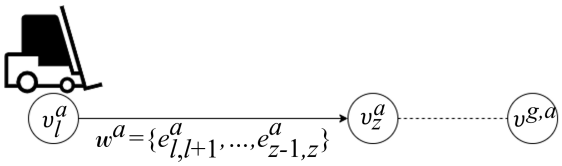


Figure 7. Allocation process for the a-th AGV, illustrating the allocated queue w^{a} from the currently occupied vertex v_{l}^{a} to the target vertex v_{z}^{a}, depicted with a solid black arrow. The remaining portion of the path π^{a} from v_{z}^{a} to $v^{g,a}$ is represented by a dashed line.

8 Path Allocator

The path allocator module acts as the execution layer of the traffic management architecture. Although the L-MAPF coordinator periodically computes collision-free trajectories, each computation requires a non-negligible processing time and is performed at fixed recomputation intervals of η timesteps. During each interval, AGVs operate in continuous time, and their motion is subject to execution uncertainties that may introduce deviations from the predicted timing.

Remark 9. Execution Uncertainties. *During execution, AGVs are exposed to both internal and external sources of uncertainty that affect their nominal traversal times. Internal factors include variations in acceleration and deceleration profiles, differences in initial conditions (e.g., entering a segment from a full stop or while already in motion), mechanical wear, and sensor or control inaccuracies. External factors arise from the operational environment and typically include vehicle alarms, safety laser scanner activations due to temporary obstacles, human interventions, and other transient disturbances. These combined effects may force an AGV to slow down or stop unexpectedly, resulting in deviations from the predicted timing assumed by the coordinator.*

As a result, two AGVs that are conflict-free in the computed space-time solution may still collide during execution if their actual traversal times deviate from the nominal ones. The path allocator eliminates this risk by regulating access to the roadmap and ensuring safe execution of the coordinated plan. Based on the trajectories $\mathcal{H}^{\star}$ provided by the L-MAPF coordinator as a reference, it assigns edges and determines the corresponding allocated trajectory and target vertex for each active AGV, enforcing mutual exclusion over all roadmap elements involved in potential conflicts so that their collision sets remain disjoint. The resulting target vertices and allocated trajectories are then supplied as inputs for the subsequent instance calculation for the L-MAPF coordinator.

As depicted in Fig. 7, the allocated edges for each a-th active AGV are contained in the allocated queue w^{a}, defined as an ordered sequence of edges $\{e_{l,l+1}^{a}, \ldots, e_{z-1,z}^{a}\}$, with $l, z \in \{1, \ldots, L^{M,a}\}$ and $l \leq z$. In particular, w^{a} connects the currently occupied vertex $v_{l}^{a} \in \pi^{a}$ to the target vertex $v_{z}^{a} \in \pi^{a}$ (see Section 7.1). From w^{a}, the allocated trajectory u^{a} is determined. Hence, each a-th active AGV is associated

Algorithm 5: Allocated Queues Updates

Input: $\epsilon, \mathcal{W}, \mathcal{U}, \mathcal{H}^\star, \mathcal{V}^z$
Output: $\mathcal{W}, \mathcal{U}, \mathcal{V}^z$

1 **foreach** $\gamma^{a,\star} \in \mathcal{H}^\star$ **do**
2 $k \leftarrow 0$
3 **while** $k <$ length$(\gamma^{a,\star})$ **do**
4 $\{v_i^a, v_j^a, \tau^a, D_{i,j}^a\} \leftarrow \gamma^{a,\star}[k]$
5 **if** $\tau^a > \epsilon$ **then**
6 break
7 **if** $v_i^a = v_j^a$ **then**
8 break
9 **if** $v_i^a = v_g^a$ **then**
10 break
11 $e_{i,i+1}^a \leftarrow GetEdge(v_i^a, v_{i+1}^a)$
12 $\mathcal{D}_{i,i+1}^a \leftarrow GetCollisionSet(e_{i,i+1}^a)$
13 $CollisionDetected \leftarrow$ false
14 **foreach** $w^b \in \mathcal{W} \setminus w^a$ **do**
15 **foreach** $e_{m,m+1}^b \in w^b$ **do**
16 **if** $e_{m,m+1}^b \in \mathcal{D}_{i,i+1}^a$ **then**
17 $CollisionDetected \leftarrow$ true
18 **if** $CollisionDetected =$ true **then**
19 break
20 Append $e_{i,i+1}^a$ to w^a
21 Append $\{v_i^a, v_{i+1}^a, \tau^a, D_{i,i+1}^a\}$ to u^a
22 $v_z^a \leftarrow v_{i+1}^a$
23 $k \leftarrow k + 1$
24 **return** $\mathcal{W}, \mathcal{U}, \mathcal{V}^z$

with an allocated queue w^a, an allocated trajectory u^a, and a target vertex v_z^a.

Remark 10. Empty Allocated Queue and Trajectory. *The allocated queue w^a and the trajectory u^a of the a-th active AGV are empty if $l = z$, i.e., the AGV is occupying its current target vertex. In case the AGV is assigned a new task, i.e., $l = 1$, w^a and u^a are initialized as empty.*

The sets of allocated queues $\mathcal{W} = \{w^a | a \in \mathcal{A}\}$, allocated trajectories $\mathcal{U} = \{u^a | a \in \mathcal{A}\}$, and target vertices $\mathcal{V}^z = \{v_z^a | a \in \mathcal{A}\}$ are updated each time a new instance is computed, as outlined in Algorithm 5 - Allocated Queues Updates, ensuring consistency and efficient edge allocation.

From the current L-MAPF coordinator instance, Algorithm 5 takes as inputs the sets $\mathcal{W}$, $\mathcal{U}$, $\mathcal{V}^z$, $\mathcal{H}^\star$, and a user-defined threshold parameter ϵ. The parameter ϵ specifies that edge allocation must be performed within a time horizon of ϵ timesteps. Ignoring ϵ can extend the allocation unnecessarily, reducing coordination efficiency and limiting the L-MAPF coordinator's ability to adjust trajectories based on updated information within the next ϵ timesteps.

Algorithm 5 iterates through each trajectory $\gamma^{a,\star} \in \mathcal{H}^\star$, processing all move and wait actions. For each action $\{v_i^a, v_j^a, \tau^a, D_{i,j}^a\}$, the algorithm performs the following checks:

- If $\tau^a > \epsilon$, the action occurs beyond the time horizon defined by ϵ (Algorithm 5, line 5).
- If $v_i^a = v_j^a$, a wait action occurs at vertex v_i^a, preventing the a-th AGV from proceeding (Algorithm 5, line 7).
- If $v_i^a = v^{g,a}$, the path allocator has already allocated edges up to the task goal vertex $v^{g,a}$ (Algorithm 5, line 9).

If any of these conditions are satisfied, the algorithm terminates the analysis of $\gamma^{a,\star}$. Otherwise, Algorithm 5 retrieves the edge $e_{i,i+1}^a$, corresponding to the move action from v_i^a to v_{i+1}^a, and its collision set $\mathcal{D}_{i,i+1}^a$ (Algorithm 5, lines 11-12).

Next, the algorithm compares $e_{i,i+1}^a$ with edges in any other allocated queue $w^b \in \mathcal{W} \setminus w^a$. If a collision is detected for at least one edge $e_{m,m+1}^b$ in w^b (i.e., $e_{m,m+1}^b \in \mathcal{D}_{i,i+1}^a$), the analysis of $\gamma^{a,\star}$ is stopped (Algorithm 5, lines 13-19). In contrast, if no collision is detected, the edge $e_{i,i+1}^a$ is added to w^a, $\{v_i^a, v_{i+1}^a, \tau^a, D_{i,i+1}^a\}$ is added to u^a, and the target vertex v_z^a is updated (Algorithm 5, lines 20-22).

Algorithm 5 then proceeds to analyse the next action of $\gamma^{a,\star}$. Once all trajectories in $\mathcal{H}^\star$ have been completely processed, the algorithm returns the updated sets of allocated steps $\mathcal{W}$, allocated trajectories $\mathcal{U}$, and target vertices $\mathcal{V}^z$.

In accordance with the VDA5050 standard, each a-th active AGV stores its currently allocated queue w^a, communicated by the TM, in a local queue. The allocation gives the AGV the right-of-way in traversing the edges contained in w^a without stopping (see Assumption 3). While moving along the allocated edges, each a-th AGV communicates its state to the TM. Upon reaching vertex v_l^a from v_{l-1}^a, the path allocator deallocates the edge $e_{l-1,l}^a$, removing it from w^a, and allows other AGVs to allocate edges within the collision set $\mathcal{D}_{l-1,l}^a$. In addition, the path allocator removes the corresponding move action from the allocated trajectory u^a and updates τ^a for all subsequent actions accordingly.

In case w^a runs empty (i.e., $v_l^a = v_z^a$), the a-th AGV stops at the current target vertex v_z^a and starts to brake beforehand.

9 Deadlock Detector and Handler

The implementation of a deadlock detection and resolution strategy is essential for minimizing AGV operational interruption. In our TM, deadlocks typically arise under two primary conditions: mainly (1) due to unpredictable events such as task updates and also (2) when the bounded horizon δ is insufficient for the ABH-CBS algorithm to resolve space-time collisions in non-corridor sectors. Deadlocks are categorized based on the nature of the precedence relationships among the involved AGVs (Pratissoli et al. 2023):

- *Cyclic Deadlocks*, where the precedence relationships between AGVs form a circular dependency.
- *Acyclic Deadlocks*, where the precedence relationship forms a directed acyclic graph structure.

As shown in Fig. 8, in this work, acyclic deadlocks typically exhibit a nested structure, where AGVs give priority to other AGVs, that are already involved in deadlock scenarios.

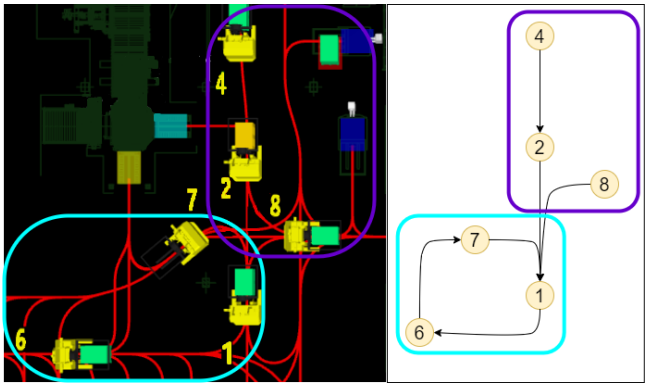


Figure 8. The figure illustrates two distinct types of deadlock scenarios. AGVs 1, 6, and 7 are involved in a circular deadlock, shown within a light blue circle, while AGVs 8, 2, and 4 are engaged in a nested acyclic deadlock, highlighted by a purple circle.

Hence, we propose the deadlock detector and handler modules, based on our previous work (Bonetti et al. 2024a), that detect deadlocks and resolve them by updating the paths of the involved AGVs.

On the one hand, the deadlock detector entails analyzing the output of the current L-MAPF coordinator instance to determine the existence of deadlocks and identifying the involved active AGVs. From the constraint set $\mathcal{M}^\star$, the deadlock detector builds the precedence graph $\mathcal{G}^P$, which models the actual precedence relationships between AGVs. The vertex set $\mathcal{V}^P(\mathcal{G}^P) = \mathcal{A}$ represents active AGVs, while the edge set $\mathcal{E}^P(\mathcal{G}^P)$ denotes their precedence relationships. Specifically, an edge (a, b) is inserted in the precedence graph if the a-th active AGV is currently occupying its target vertex v_z^a, i.e., $\tau_z^a = 0$ (see Remark 7), and a constraint $\mu^a = (a, v_i^a, v_{i+1}^a, [\tau^I, ..., \tau^F], b) \in \mathcal{M}^\star$ exists to resolve a space-time collision with the b-th AGV, such that:

- $v_i^a = v_z^a$, meaning that the constraint refers to an action starting from the current target vertex v_z^a of the a-th AGV.
- $\tau^I \leq D_{i,i+1}^a$, meaning that the a-th AGV cannot perform the move action from the target vertex to the next vertex v_{z+1}^a.

After iterating over all $\mathcal{M}^\star$, the deadlock detector obtains the complete precedence graph $\mathcal{G}^P$ relative to the current L-MAPF coordinator instance. Then, as described in Bonetti et al. (2024a), the deadlock detector identifies cyclic deadlocks employing the widely used Depth-First Search (DFS) algorithm (Cormen et al. 2001), a common technique for detecting deadlocks in operating systems. The DFS algorithm begins from an initial node in the graph and recursively explores each branch as deeply as possible before backtracking to previously visited nodes. This traversal method enables DFS to efficiently navigate through graph structures while maintaining a record of visited nodes, typically in a dedicated list (Venkatesh et al. 1998). A cycle is detected when the algorithm encounters a node that has already been visited within the same recursive path, indicating the presence of a deadlock. In our case, the DFS algorithm specifically identifies AGVs involved in precedence cycles within $\mathcal{G}^P$. By merging all the AGVs contained in precedence cycles, the deadlock detector obtains the set $\mathcal{B}$ containing the AGVs that require deadlock resolution due to cyclic deadlocks.

Algorithm 6: Acyclic Deadlock Detection

Input: $\mathcal{B}, \mathcal{G}^P$
Output: $\mathcal{B}$
1 **foreach** $(a, b) \in \mathcal{E}^P(\mathcal{G}^P)$ **do**
2 **if** $a \notin \mathcal{B}$ **and** $b \in \mathcal{B}$ **then**
3 $\mathcal{B} \leftarrow \mathcal{B} \cup \{a\}$
4 $\mathcal{B} \leftarrow$ **Algorithm 6**$(\mathcal{B}, \mathcal{G}^P)$
5 **return** $\mathcal{B}$
6 **return** $\mathcal{B}$

Then, differently from Bonetti et al. (2024a), the deadlock detector also addresses acyclic nested deadlocks using Algorithm 6 - Acyclic Deadlock Detection. The procedure iterates over the edge set $\mathcal{E}^P(\mathcal{G}^P)$, checking if there exists an edge (a, b) such that $a \notin \mathcal{B}$ and $b \in \mathcal{B}$ (Algorithm 6, lines 1-2). Upon detecting an edge (a, b) that meets these conditions, node a is added to the set $\mathcal{B}$, and Algorithm 6 is invoked recursively (Algorithm 6, line 4). The process ultimately returns the complete set $\mathcal{B}$ of AGVs involved in deadlocks that require path replanning.

The idea of representing inter-AGV dependencies through a graph shares a conceptual similarity with the Action Dependency Graph (ADG) and the Temporal Plan Graph (TPG) used in Simple Temporal Networks (STN) approaches (Hönig et al. 2019; Hönig et al. 2016). In our case, however, the precedence graph employed for deadlock detection captures only the immediate precedence relations between AGVs in the current L-MAPF coordinator instance, and therefore does not model the full temporal evolution of dependencies characteristic of STN-based formulations.

On the other hand, once the set $\mathcal{B}$ has been defined, the deadlock handler employs the CBS algorithm (Sharon et al. 2015), which is both optimal and complete, to solve the deadlocks. Completeness is crucial in deadlock resolution, as it guarantees that if a feasible set of conflict-free trajectories exists for the involved AGVs, the algorithm will eventually find it, ensuring system recovery from any solvable blocking condition. In particular, the CBS algorithm calculates a new path for each AGV in $\mathcal{B}$ to ensure that ABH-CBS can perform the AGV coordination without any blockages.

Although CBS can be computationally intensive (Gordon et al. 2021), this is not a significant concern for deadlock resolution in our AGV system. The number of AGVs involved in deadlocks is typically small, and computational constraints are less critical since the AGVs remain stationary during the resolution process.

Specifically, our CBS employs the Multi-Label A* (Grenouilleau et al. 2019b) as the low-level planner to calculate the paths from v_z^a to $v^{g,a}$ and from $v^{g,a}$ to $v^{g',a}$. This approach aligns with the path definition outlined in Section 6. Once a new path for each a-th AGV in $\mathcal{B}$ has been computed, the deadlock handler updates the respective paths in $\mathcal{P}$. Subsequently, the sets $\mathcal{K}^{\text{tot}}$ and $\mathcal{S}$ are adjusted accordingly to ensure that the next instance of the L-MAPF coordinator is correctly initialized (see Section 7.1).

Although the proposed deadlock detector and handler enables automatic recovery from blocking situations, failures may still occur in rare cases where the roadmap layer does not offer the segments required for a feasible reconfiguration of the involved AGVs. Such situations typically arise in highly constrained areas of the plant, where the directed and predefined graph (see Assumption 1) provides no admissible detour or backtracking maneuver within the physical layout. When the resolution phase fails under these conditions, the system escalates the issue, requiring human operators to temporarily switch the affected AGVs to manual mode and reposition them to restore operability.

10 Case Study

This section describes the experimental setup (see Section 10.1) and the results (see Section 10.2) of the proposed traffic management system. In particular, our algorithm is developed in collaboration with *Gruppo TecnoFerrari S.p.A.*, an Italian company specializing in end-of-line machinery, with a focus on warehouse and plant logistics using AGVs.

The solution is implemented in C#, integrated into the TecnoFerrari Supervisor software for AGV systems, which includes the Task Manager module. All experiments are conducted on a notebook equipped with an Intel Core i7-1165G7 processor (2.80 GHz clock speed) and 16 GB of DDR4-3200 RAM, and are validated in a real industrial environment in collaboration with the company (see Fig. 9). The video attached to the supplementary material (see Extension 1) shows some of the tests conducted in the automated factory.

10.1 System Setup

The effectiveness and adaptability of the traffic management system are evaluated using three realistic layouts that simulate a palletizing, storage, and pallet-wrapping plant located at the end of a production line. The corresponding views in the TecnoFerrari Supervisor software are shown in Fig. 10, Fig. 11, and Fig. 12. The roadmaps and plant layouts, provided by *Gruppo TecnoFerrari S.p.A.*, are used to generate the roadmap layer, ensuring compatibility with different AGV classes and their operational constraints. Based on this topological information, the topological layer is constructed by partitioning each plant into sectors and identifying corridor locations.

The three layouts represent small-, medium-, and large-scale industrial environments relevant to palletizing, storage, and pallet-wrapping plants. The small environment is a non-standardized layout characterized by narrow dead-end bidirectional corridors and a tightly interconnected routing structure that restricts the AGV's feasible path options. The structural constraints of the small layout inherently increase the likelihood of congestion and deadlock formation, making this environment representative of challenging coordination conditions for large AGVs. The medium environment preserves the non-standardized structure but is slightly larger, with wider operational areas and a broader set of interconnected sectors. Traffic density increases due to the presence of a larger fleet, while irregular geometry, asymmetric intersections, and dead-end bidirectional corridors continue to produce

Figure 9. A photo taken during the experiments conducted in the automated factory. The AGVs are coordinating in narrow and non-standardized settings.

complex coordination conditions. This layout also includes heterogeneous AGVs, showcasing the capability of the proposed solution to handle multiple AGV classes operating simultaneously. The large environment corresponds to a more regular facility where heterogeneous AGVs move only along unidirectional tracks, resulting in a simpler navigation structure compared to the bidirectional corridor configurations of the smaller layouts. Although not central to the study, this layout is included to demonstrate the robustness and general applicability of the proposed traffic management system when coordinating a larger fleet in a nearly standardized layout.

To quantitatively characterize the topological and geometric differences between the three layouts, two evaluation metrics are computed from the graphs associated with the corresponding roadmaps.

The first metric is the *Origin-Destination Betweenness Centrality* (O-D BC), a topological indicator from network science (Chen et al. 2023a) that measures how frequently an edge lies on the shortest paths connecting any pair of origin-destination vertices. For two generic starting v^s and goal v^g vertices representing station or battery-charger locations, let $N^e_{v^s,v^g}$ denote the number of shortest paths from v^s to v^g that traverse edge e, and let N_{v^s,v^g} be the total number of shortest paths between them. The O-D BC of edge e is defined as:

$$\text{O-D BC}(e) = \sum_{(v^s,v^g)} \frac{N^e_{v^s,v^g}}{N_{v^s,v^g}}.$$

This metric is evaluated over the roadmap layer to quantify the structural relevance of each edge.

Higher O-D BC values identify edges that play a critical role in traffic flow and are more likely to act as bottlenecks during AGV operation. For each layout's roadmap layer, the mean, variance, and standard deviation of the normalized O-D BC values are computed to characterize the distribution of centrality and to highlight regions where increased flow intensity may occur.

The second metric is the *Graph Density Index* (GDI) (Menniti et al. 2013), defined for directed graphs without loops as:

$$\text{GDI} = \frac{|\mathcal{E}^R|}{|\mathcal{V}^R|(|\mathcal{V}^R|-1)}$$

where $|\mathcal{E}^R|$ denotes the number of directed edges and $|\mathcal{V}^R|$ the number of vertices in the roadmap layer. GDI

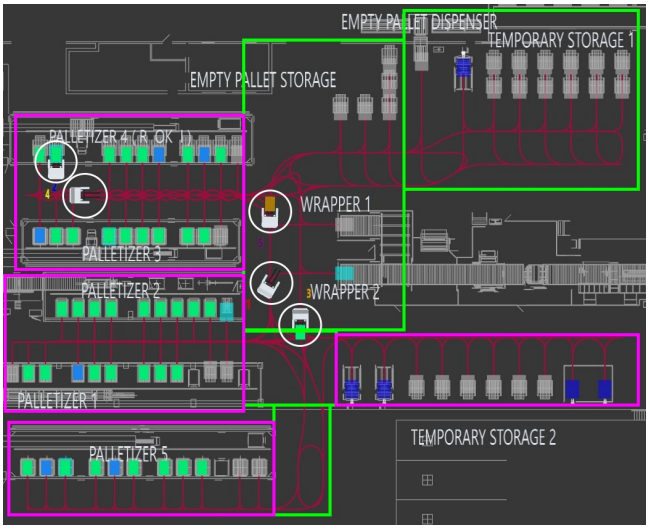


Figure 10. 2D reconstruction of the plant in the first layout, with green areas delineating non-corridor sectors and magenta areas representing dead-end narrow corridors, forming a narrow, non-standardized environment, where AGVs of class C_1, marked by a white circle, are in motion.

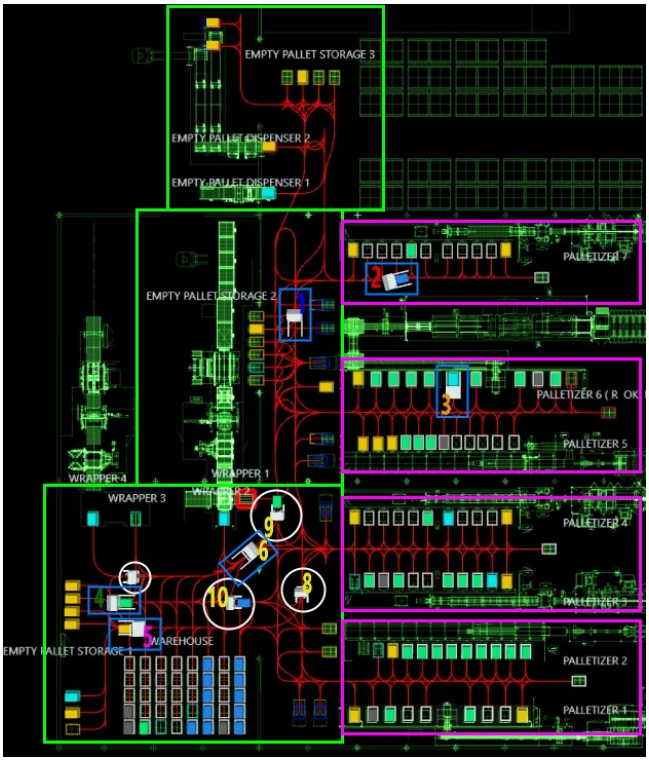


Figure 11. 2D reconstruction of the plant in the second layout, with green areas delineating non-corridor sectors and magenta areas representing dead-end corridors, forming a medium-sized, high-traffic density, and non-standardized environment, where AGVs of class C_1 and class C_2, marked by a white circle and a blue square, respectively, are in motion.

quantifies the overall density of connections in the roadmap graph. In our environments, higher GDI values indicate a larger number of directed connections, primarily due to the presence of bidirectional corridors. The increased connectivity results in layouts that are more demanding from a coordination perspective.

The indices are evaluated for the three layouts and reported in Table 3, where the first column identifies the layout, and the remaining columns report the BC and GDI indices.

The first layout, depicted in Fig. 10, represents a narrow, non-standardized environment where AGVs transport pallets from the palletizers to the wrapping machine, redirecting them to temporary storage when the wrapping machine is occupied or unavailable. Empty pallets are supplied via a dispenser, enabling the AGVs to restock the palletizers whenever required. The facility, with horizontal dimensions of 60×40 m^2, is divided into 8 sectors, including 4 corridors. The homogeneous AGVs, belonging to class C_1, have a rectangular footprint of 2.9×1.6 m^2 and are designed to navigate the confined layout efficiently, with a maximum speed of 1 m/s. Effective task coordination is essential to maintain a continuous workflow and to limit delays in such a constrained environment. Table 3 reports an O-D BC mean value of 0.281, a BC variance of 0.015, and a BC standard deviation of 0.122 for Layout 1. The values indicate that many origin-destination shortest paths tend to converge on a limited set of edges, producing a highly concentrated flow pattern and increasing the likelihood of bottleneck formation. The GDI value of 0.010 reflects a relatively high level of connectivity, largely attributable to the presence of bidirectional corridors, which introduce a greater number of edges despite the overall constrained layout geometry.

The second layout in Fig. 11 models a medium-sized, non-standardized environment to assess the system's scalability under high-traffic density. The facility of horizontal size 50×70 m^2 is also divided into 7 sectors, including 4 corridors. The AGV fleet consists of heterogeneous vehicles categorized into two classes: class C_1, and class C_2, with a

Table 3. Layout evaluation metrics.

Layout	O-D BC			GDI
	Mean	Var	Std Dev	
1	0.281	0.015	0.122	0.010
2	0.187	0.012	0.118	0.005
3	0.131	0.016	0.134	0.002

rectangular footprint of 3.2×1.8 m^2, both with a maximum speed of 1 m/s. The AGVs are tasked with transporting pallets from the palletizers to the wrapping machine. In cases where the wrapping machine is occupied or unavailable, the AGVs transport pallets to a designated warehouse area. Furthermore, empty pallets are supplied from two dispensers, enabling the AGVs to restock the palletizers when required or deposit them in the empty pallet storages. Table 3 reports an O-D BC mean value of 0.187, a variance of 0.012, and a standard deviation of 0.118 for Layout 2. The values indicate a less concentrated centrality distribution compared to the small layout, reflecting the presence of wider areas and a routing structure that is less tightly interconnected, which allows shortest paths to distribute more evenly across the network. The GDI value of 0.005 suggests a moderate level of connectivity, lower than in Layout 1 due to the larger graph size but still aligned with the non-standardized structure and the bidirectional corridor pattern of the environment.

The third layout in Fig. 12 evaluates the system's performance in a large environment of horizontal size 210×115 m^2 featuring no corridor sectors. Although large and

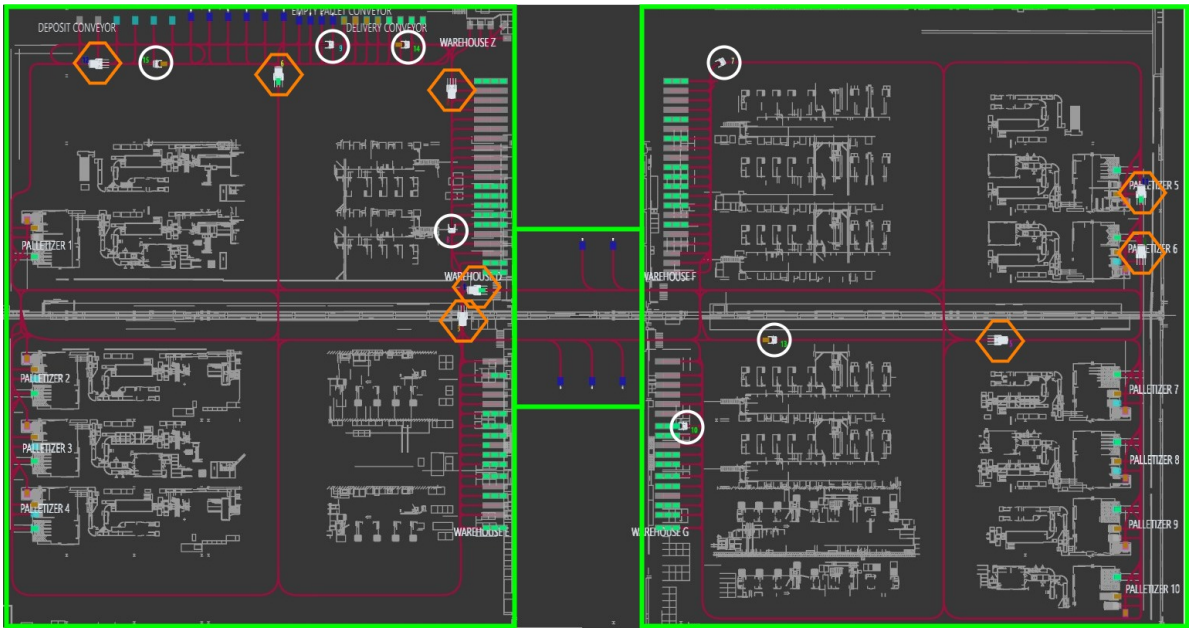


Figure 12. 2D reconstruction of the plant in the third layout, representing a large environment divided by green areas that delineate non-corridor sectors, where AGVs of class C_2 and C_3, marked by a blue square and an orange hexagon, respectively, are in motion.

Table 4. Parameters set during the testing phase for all three layouts.

Parameter	Description	Layout 1	Layout 2	Layout 3
τ	Timestep	1 [s]	1 [s]	1 [s]
η	Replanning Time	1 τ	1 τ	1 τ
δ'	Base Time Horizon	30 τ	30 τ	30 τ
δ''	Horizon Increment	10 τ	10 τ	10 τ
t^σ	Timeout	250 [ms]	500 [ms]	500 [ms]
ϵ	Allocation Horizon	6 τ	6 τ	10 τ

unconstrained facilities are not the primary focus of the study, this scenario offers insight into system behavior and performance under more regular and less restrictive operating conditions. In particular, the layout includes AGVs belonging to class C_3 with a rectangular footprint of 4.3×1.5 m^2 and a maximum speed of 0.8m/s, operating alongside AGVs of class C_1. The AGVs transport pallets from the palletizers or the deposit conveyor to the warehouses or delivery conveyor, and also provide palletizers with empty pallets from the empty pallet conveyor. Table 3 reports an O-D BC mean value of 0.131, a variance of 0.016, and a standard deviation of 0.134 for Layout 3. The values indicate a generally lower centrality level compared to the smaller layouts, as shown by the reduced mean, while the relatively high standard deviation reveals that certain edges are traversed by substantially more shortest paths than the rest of the roadmap. The GDI value of 0.002 reflects the low graph density resulting from the use of unidirectional tracks, which provide fewer edges than the bidirectional corridors found in the smaller layouts.

Table 4 summarizes the parameters adopted across the three layouts. The timestep τ is set to 1 s (Row I), a choice aligned with common practice in AGV traffic management systems (Pratissoli et al. 2023). A one-second discretization matches typical AGV speed, offers adequate temporal resolution for conflict detection and avoidance, and limits unnecessary computational overhead. The replanning time η is set to 1 τ (Row II), enabling the L-MAPF coordinator to react to execution uncertainties at every timestep. This configuration corresponds to a common update frequency used in industrial AGV control systems (Liu et al. 2021). The base time horizon δ' is set to 30 τ (Row III), while the horizon increment δ'' is set to 10 τ (Row IV). The initial horizon provides sufficient look-ahead for capturing short-term interactions in congested areas as shown in our previous work (Bonetti et al. 2024b), whereas the incremental extension balances computational effort and solution refinement within the anytime behavior of the L-MAPF coordinator. The timeout t^σ values (Row V) are chosen according to the operational requirements of the three facilities and the computational constraints imposed by the TecnoFerrari Supervisor software, which executes concurrently with the traffic management system. The allocation horizon ϵ (Row VI) is selected to match the scale and structural characteristics of each layout. The third layout adopts a larger value than the first two (Columns III-IV), reflecting the longer traversal distances and extended queues associated with its wider geometry. Among all parameters, ϵ is the only one not fixed by industrial conventions, hardware limits, or real-time operational requirements. It directly modulates the interaction between the path allocator and the L-MAPF coordinator by determining the portion of each AGV's path reserved during execution, a feature introduced by the proposed system and central to the balance between workspace allocation and execution flexibility. A detailed assessment of the impact of ϵ is presented in the sensitivity analysis reported in Section 10.2.2.

10.2 Results

To evaluate the effectiveness of the proposed approach, we define a structured set of scenarios for the three layouts

described above. For each layout, scalability is assessed through a first group of scenarios (1.A-1.C, 2.A-2.D, and 3.A-3.D), where the fleet size is progressively increased to determine the configuration that maximizes throughput. Selecting throughput-maximizing configurations aligns with the primary operational goal of AGV systems, namely completing tasks at the highest possible rate under realistic operating conditions, and identifies the fleet sizes that exploit each layout's capacity most effectively before congestion becomes dominant.

Based on the scalability results, the scenarios producing the highest throughput for each layout (1.B, 2.B, and 3.B) are selected as reference cases for three additional analyses. The first analysis, a sensitivity study, evaluates the impact of the allocation horizon ϵ, a concept introduced in the present work, on system performance. The second analysis, an ablation study, evaluates the contribution of anytime and EHC strategies included in the proposed L-MAPF coordinator. The final analysis, a comparative evaluation, benchmarks the proposed solution against three alternative coordination methods: a traditional rule-based traffic management system commonly used in logistics operations, a state-of-the-art industrial solution, and a priority-based L-MAPF method.

To assess the performance of the algorithm across the scenarios, we consider the following Key Performance Indicators (KPIs):

- **Throughput:** the average number of tasks completed per hour by the AGVs. A higher throughput value indicates improved traffic management system efficacy, as it reflects a greater number of tasks being executed within the same time span.
- **Average Flow Time:** the average time required to accomplish a task in the system, from allocation to completion. A lower average flow time indicates a more efficient system, as tasks are completed more quickly, reducing delays and improving overall responsiveness.
- **Management Efficiency:** the traffic management efficiency is defined as $\frac{T^M}{T^M+T^W}$, where T^M represents the time AGVs spend actively moving, while T^W is the time spent waiting. This metric quantifies the proportion of operational time dedicated to AGV movement. A higher efficiency value indicates a more effective coordination strategy, minimizing idle time and maximizing productive AGV usage.
- **Number of Deadlocks:** the number of deadlocks generated during testing phase. A lower number of deadlocks indicates a more robust coordination strategy, ensuring smoother traffic flow and minimizing disruptions in the system.
- **Deadlocks Detected & Resolved Percentage:** the percentage of deadlocks effectively detected and resolved by the deadlock detector and handler modules. A higher percentage indicates a more effective resolution strategy, ensuring minimal system disruptions and maintaining continuous AGV operations.
- **Average Deadlock Resolution Time:** the average computational time required to solve deadlocks. A lower resolution time indicates a more efficient deadlock handling mechanism, enabling quicker recovery and minimizing delays in AGV operations.
- **Average Time Window (TW)**: the average value of δ reached during ABH-CBS calculation. It represents the computational effort required to solve an instance. Lower values of Average TW indicate a higher computational complexity.
- **Valid Solution Percentage:** the percentage of ABH-CBS instances for which a solution is found before the computational time expires. A higher percentage reflects a more reliable and efficient algorithm, ensuring timely coordination.

In the ablation study, the KPIs associated with features specific to ABH-CBS (Average TW and Valid Solutions Percentage) are not reported, since the ablated variants do not implement anytime refinement or timeout-based termination. The same applies to the comparative analysis, where the baseline coordination methods do not include anytime expansion or a timeout stopping rule, making these metrics inapplicable. Deadlock-related KPIs are also reported only for approaches that explicitly incorporate deadlock detection and resolution mechanisms.

The data required to evaluate the KPIs is collected during execution within the TecnoFerrari Supervisor software, where AGVs operate under continuous task assignment. Each scenario runs for approximately ten hours, providing an observation period long enough to capture representative operating conditions and to obtain statistically meaningful performance indicators under sustained workload.

A dedicated convergence analysis is reported at the end of the section. The study focuses on Layout 1 with the fleet size of scenario 1.B, a configuration that maximizes throughput while keeping the computational load sufficiently moderate to allow systematic data collection. The selected operating point offers a representative and tractable setting for examining the anytime behavior of ABH-CBS and for quantifying its progression toward the full-horizon CBS solution. The analysis also monitors the evolution of solution quality as a function of cumulative elapsed time, providing a complementary perspective on the convergence dynamics of the solver.

10.2.1 Scalability Analysis To assess scalability, performance is evaluated across the three layouts while progressively increasing the fleet size. The complete set of KPIs is reported in Table 5.

Across all layouts, throughput rises as additional AGVs are introduced, confirming that the proposed traffic management system can effectively exploit larger fleets even under growing traffic density. Management efficiency, however, steadily decreases: with more vehicles operating simultaneously, a larger share of time is spent waiting rather than moving. The resulting increase in flow time indicates the higher coordination effort imposed by denser operating conditions.

Each layout eventually reaches a saturation point where the negative effects of congestion outweigh the benefits of

Table 5. KPI values obtained for the three layouts in the scalability analysis, as the number of AGVs N^A varies.

Scenario	N^A	Throughput [h^{-1}]	Average Flow Time [s]	Management Efficiency	No. Deadlocks	Deadlocks Detected & Resolved Percentage [%]	Average Deadlock Resolution Time [ms]	Average TW [τ]	Valid Solutions Percentage [%]
1.A	3	137	77	0.93	1	100	144	151.37	100
1.B	5	208	85	0.84	3	100	378	112.80	100
1.C	7	189	127	0.56	8	75	617	70.00	97.98
2.A	5	178	100	0.94	0	–	–	132.64	100
2.B	8	246	115	0.82	1	100	1765	80.93	99.82
2.C	10	283	125	0.75	4	75	1828	60.97	98.22
2.D	12	242	178	0.53	13	85	3256	43.64	28.65
3.A	10	195	181	0.93	0	–	–	130.74	100
3.B	15	288	183	0.92	0	–	–	100.38	99.97
3.C	20	341	204	0.82	1	100	269	72.16	87.42
3.D	25	327	265	0.63	4	100	304	47.41	43.18

additional AGVs. In Layouts 1 and 2, the threshold appears in 1.C and 2.D: waiting times escalate, local congestion becomes persistent, and throughput no longer increases. This behavior results from the restricted routing options available in the two layouts and from the growth in coordination complexity induced by the increasing number of inter-agent interactions. Layout 3 follows the same qualitative trend, but the saturation point appears at a larger operational scale due to the wider geometry and the presence of unidirectional lanes. Throughput increases up to scenario 3.C; in 3.D, however, the pronounced rise in flow time and the reduction in management efficiency indicate that both the structural capacity of the layout and the computational load on the coordinator have reached their limits. Beyond that point, adding more AGVs no longer enhances productivity and instead intensifies congestion.

Deadlock occurrence increases consistently with fleet size across all layouts, reflecting the limited routing flexibility typical of non-standardized industrial environments. The deadlock detection and resolution module handles the vast majority of events, and no undetected deadlocks are observed. Only the most overloaded scenarios (1.C and 2.C-2.D) show a reduction in the fraction of resolved cases (down to 75-85%), largely because the involved AGVs lack any feasible motion to resolve the blockage. Resolution times increase with fleet size, as a higher number of AGVs makes multi-vehicle deadlocks more likely, which naturally requires longer coordination and resolution effort.

For ABH-CBS, the average TW decreases as the fleet becomes larger. An increase in the number of agents raises both the number of potential conflicts and the amount of constraint checking, directing most of the available computation toward resolving imminent interactions rather than extending the base horizon δ'. The effect is particularly evident in 2.D and 3.D, where TW drops to values close to δ'. A similar pattern emerges in the valid solutions percentage: in the most congested scenarios, the solver often fails to reach δ' before the timeout, leading to the lowest recorded rates (28.65% and 43.18%). In all other scenarios, TW remains sufficiently large to sustain reliable conflict resolution, and the valid solutions percentage stays consistently high even under significant traffic density, confirming the robustness of the proposed approach when operating under real-time constraints.

Across the scalability analysis, 1.B, 2.C, and 3.C emerge as the throughput-maximizing configurations for Layouts 1, 2, and 3 respectively. Each configuration represents the fleet size that exploits the corresponding layout capacity most effectively while remaining below the congestion threshold reached in more populated settings. The three scenarios act as the reference operating points for the sensitivity, ablation, and comparative evaluations.

10.2.2 Sensitivity Analysis A sensitivity analysis is conducted to assess how the allocation horizon ϵ influences the performance of the proposed traffic management system. Three representative values are examined to capture different operating regimes: a short horizon ($\epsilon = 4$), a medium horizon ($\epsilon = 6$), and a long horizon ($\epsilon = 10$). The selected values allow us to evaluate how the parameter used by the path allocator to reserve portions of the roadmap affects coordination performance. The results across all layouts are reported in Table 6.

For Layouts 1 and 2, results show that $\epsilon = 4$ and $\epsilon = 6$ lead to nearly identical values of throughput, average flow time, and management efficiency. The only marked difference concerns ABH-CBS: with $\epsilon = 6$, the solver attains a slightly larger average TW, improving predictive performance without altering the KPIs. As a result, $\epsilon = 6$ emerges as the most balanced configuration in the two layouts. The improvement is driven by the longer reserved portion of the path, which shifts the starting timestep of the space-time search forward and enables deeper exploration of the temporal dimension within the same timeout.

Increasing the allocation horizon to $\epsilon = 10$ in Layouts 1 and 2 leads to a slight but consistent degradation in throughput, average flow time, and management efficiency, even though the average TW becomes larger. The decline arises from the longer portion of path allocated to each AGV: an overly extended reservation of segments reduces flexibility and amplifies the effect of execution uncertainties. When an AGV slows down because of an unexpected obstacle, the roadmap element belonging to the collision set of the allocated segment remains unavailable to other AGVs for an unexpectedly long time. Since the allocator enforces non-overlapping allocations, the remaining AGVs must wait until the segment is released, which negatively affects overall coordination performance when execution uncertainties occurs. Across both layouts, the number of deadlocks remains comparable for all values of ϵ, indicating that the allocation horizon primarily affects solution quality rather than the structural likelihood of deadlock formation.

In Layout 3, all tested values of ϵ lead to nearly identical operational performance, reflecting the standardized geometry and predominantly unidirectional tracks. In this setting,

Table 6. KPI values obtained for the three layouts in the sensitivity analysis, with the fleet size set to $N^A = 5$ for scenarios 1.B.a–1.B.b–1.B.c, $N^A = 10$ for scenarios 2.C.a–2.C.b–2.C.c, and $N^A = 20$ for scenarios 3.C.a–3.C.b–3.C.c.

Scenario	ϵ	Throughput [h^{-1}]	Average Flow Time [s]	Management Efficiency	No. Deadlocks	Deadlocks Detected & Resolved Percentage [%]	Average Deadlock Resolution Time [ms]	Average TW [τ]	Valid Solutions Percentage [%]
1.B.a	4	207	85	0.84	3	100	543	111.25	100
1.B.b	6	208	85	0.84	3	100	378	112.80	100
1.B.c	10	199	89	0.80	4	100	235	116.62	100
2.C.a	4	283	124	0.75	5	80	1277	57.35	96.50
2.C.b	6	283	125	0.75	4	75	1828	60.97	98.22
2.C.c	10	276	128	0.73	3	100	1442	66.06	99.34
3.C.a	4	341	204	0.82	1	100	157	68.51	86.08
3.C.b	6	340	205	0.82	0	-	-	69.13	86.41
3.C.c	10	341	204	0.82	1	100	269	72.16	87.42

Table 7. KPI values obtained for the three layouts in the ablation study, with the fleet size set to $N^A = 5$ for scenarios 1.B.b–1.B.d–1.B.e, $N^A = 10$ for scenarios 2.C.b–2.C.d–2.C.e, and $N^A = 20$ for scenarios 3.C.c–3.C.d–3.C.e.

Scenario	Approach	Throughput [h^{-1}]	Average Flow Time [s]	Management Efficiency	No. Deadlocks	Deadlocks Detected & Resolved Percentage [%]	Average Deadlock Resolution Time
1.B.b	ABH-CBS with EHC	208	85	0.84	3	100	378
1.B.d	BH-CBS with EHC	201	88	0.81	5	100	286
1.B.e	BH-CBS	184	96	0.74	17	94	547
2.C.b	ABH-CBS with EHC	283	125	0.75	4	75	1828
2.C.d	BH-CBS with EHC	272	130	0.72	7	86	1901
2.C.e	BH-CBS	241	148	0.63	26	85	2362
3.C.c	ABH-CBS with EHC	341	204	0.82	1	100	269
3.C.d	BH-CBS with EHC	338	206	0.81	2	100	76
3.C.e	BH-CBS	339	206	0.81	2	100	257

selecting $\epsilon = 10$ does not reduce throughput or flow time and even provides a computational advantage, as ABH-CBS attains a larger average TW without compromising coordination quality. Deadlock occurrences remain similarly low for all configurations, indicating that the allocation horizon has limited influence when vehicle circulation is largely unconstrained.

10.2.3 Ablation Study To isolate the contribution of the key coordination mechanisms embedded in our L-MAPF coordinator, an ablation study is conducted across the three layouts using the throughput-maximizing scenarios, i.e., 1.B, 2.C, and 3.C. The full version of our coordinator, denoted as ABH-CBS, augments the standard Bounded-Horizon CBS (BH-CBS) with two additional mechanisms: (i) an anytime expansion of the time window, which incrementally increases δ starting from the base value δ'; and (ii) the EHC strategy, which extends the horizon δ^a for each a-th AGV as long as it operates inside narrow corridor sectors.

To quantify the impact of both mechanisms, we compare the ABH-CBS against two simplified variants. The first variant, BH-CBS + EHC, disables the anytime mechanism; in this configuration the time window cannot grow beyond the base horizon δ' in non-corridor sectors, while the EHC module remains active. As a result, the coordinator behaves as a BH-CBS outside corridors and retains full conflict-resolution capability inside corridor sectors. The second variant, BH-CBS, removes both the anytime expansion and the EHC mechanism forcing the coordinator to operate in all sectors with the base bounded horizon δ'. This configuration represents the minimal baseline and corresponds to a classical BH-CBS without any form of horizon refinement or corridor extension.

The results in Table 7 indicate that the effect of the two ablations depends strongly on the layout. In Layout 1 and Layout 2, which are narrow, non-standardized, and constrained by limited maneuvering space, disabling the anytime strategy already leads to a clear performance reduction, with throughput decreasing by 3% and 4% in 1.B.d and 2.C.d respectively. Without horizon expansion, the L-MAPF coordinator cannot anticipate conflicts beyond δ' in non-corridor sectors, so conflicts that ABH-CBS would detect earlier are identified only when the AGVs are already close to, or partially committed to, a congested region. The delay in conflict identification increases the likelihood of deadlocks, extends waiting times, and leads to noticeable degradation in throughput, average flow time, and management efficiency. In Layout 3 the impact is negligible, because the standardized geometry and the absence of narrow corridors allow the coordinator to predict conflicts effectively even when operating with the base horizon δ'.

When both the anytime strategy and the EHC mechanism are disabled, the degradation becomes substantially more severe in layouts containing narrow bidirectional corridors. With only the base horizon available, the L-MAPF coordinator cannot capture the temporal evolution of conflicts inside corridor sectors. As a result, AGVs often enter the same corridor before the impending conflict becomes detectable, leading to a large number of corridor deadlocks. The deadlock detector and handler resolve all such cases, but recovery is costly: once a deadlock forms inside a narrow corridor, the involved AGVs must stop, wait for the conflict to be processed, and perform the maneuvers required to restore feasibility before resuming their tasks. The resulting increase in travel distance and waiting time causes marked reductions in throughput and management efficiency. In Layout 1, where several narrow bidirectional corridors connect the palletizers to the rest of the plant, the absence of EHC causes corridor deadlocks to form systematically, generating repeated interruptions and costly recovery maneuvers. Compared to 1.B.b, throughput decreases by about 12% in 1.B.e. The effect is even

Table 8. KPI values obtained for the three layouts in the comparative evaluation across all tested traffic-management approaches.

Scenario	Approach	Throughput [h^{-1}]	Average Flow Time [s]	Management Efficiency	No. Deadlocks	Deadlock Detector & Handler	Deadlocks Detected & Resolved Percentage [%]
1.B.b	*ours*	208	85	0.84	3	✓	100
1.B.f	*company*	188	94	0.76	4	✗	-
1.B.g	Pratissoli et al. (2023)	194	92	0.78	4	✓	50
1.B.h	L-MAPF coordinator with PBS	196	91	0.79	7	✗	-
2.C.b	*ours*	283	125	0.75	4	✓	75
2.C.f	*company*	255	139	0.67	5	✗	-
2.C.g	Pratissoli et al. (2023)	264	132	0.71	8	✓	37
2.C.h	L-MAPF coordinator with PBS	258	138	0.68	11	✗	-
3.C.c	*ours*	341	204	0.82	1	✓	100
3.C.f	*company*	338	206	0.81	1	✗	-
2.C.g	Pratissoli et al. (2023)	337	208	0.80	1	✓	100
3.C.h	L-MAPF coordinator with PBS	338	207	0.81	2	✗	-

more pronounced in Layout 2, where higher traffic density increases both the likelihood and the complexity of corridor conflicts, further degrading system throughput in 2.C.e of approximately 15% compared to 2.C.b. By contrast, Layout 3 is essentially unaffected: the environment contains no narrow bidirectional corridors and traffic flows are predominantly unidirectional, so conflicts remain easy to anticipate and handle even with the base horizon δ', and the performance of the ablated configuration remains comparable to that of the full ABH-CBS coordinator.

10.2.4 Comparative Evaluation To assess the effectiveness of the proposed approach, we evaluate its performance against three representative baselines covering commonly adopted traffic management strategies in industrial AGV systems. The baselines consist of: (i) the rule-based TM deployed by *Gruppo TecnoFerrari S.p.A.*; (ii) the state-of-the-art TM, i.e., the solution proposed in Pratissoli et al. (2023); and (iii) the priority-based TM, obtained by replacing ABH-CBS with Priority-Based Search (PBS) (Ma et al. 2019) within the L-MAPF coordinator. The comparison is conducted on 1.B, 2.C, and 3.C, as identified in the scalability analysis. Search-based coordination methods surveyed in Section 2.1 are not included as baselines. Their idealized assumptions, i.e., unit-time actions, single-intersection layouts with symmetric unidirectional lanes, and simplified geometric models, do not match the industrial layouts considered in this work, where AGVs follow fixed paths with heterogeneous traversal times.

The rule-based TM is included as a benchmark representative of conventional industrial coordination practice. Coordination is governed by a reactive First-Come-First-Served (FCFS) reservation strategy over roadmap segments (Azimi et al. 2010), where each AGV requests access to the next segments of its path and proceeds only if the segments are not in collision with elements of the roadmap layer already reserved for other AGVs. The FCFS mechanism is augmented with handcrafted rule-based control zones that encode localized access constraints for structurally critical areas such as narrow corridors, machine entries, or intersections. Within these zones, standard FCFS reservations may be overridden by deterministic right-of-way rules, stopping conditions, or direction-dependent permissions to ensure safe traversal and avoid blocking situations. The approach reflects a family of rule-driven coordination strategies designed for deterministic execution and straightforward deployment rather than predictive reasoning or multi-agent optimization. Its inclusion in the study provides a reference point for assessing the impact of replacing prescriptive rules with search-based coordination in non-standardized and spatially constrained environments.

The state-of-the-art TM introduced in Pratissoli et al. (2023) constitutes, to the best of our knowledge, the only existing solution in the literature that can be deployed in non-standardized real industrial environments with high traffic density, where AGVs operate on a predefined roadmap composed of segments with heterogeneous traversal times. The method integrates a traffic-aware hierarchical path planner with a coordination module based on a time-expanded graph that regulates agent interactions through predictive collision avoidance and precedence assignment. Since the original formulation does not support heterogeneous AGVs, an adaptation is implemented to align with the constraints of the evaluated layouts. In particular, path generation is restricted to roadmap elements admissible for each vehicle class, and replanning is invoked exclusively during task allocation and deadlock resolution to comply with the fixed-path assumption adopted throughout the experiments.

The priority-based TM isolates the contribution of the search component within the L-MAPF coordinator of our TM by replacing ABH-CBS with PBS, a two-level priority-based planner that resolves conflicts exclusively through consistent priority assignments. At the high level, PBS evaluates alternative priority assignments; at the low level, agents are planned sequentially according to the resulting priority structure, with lower-priority agents adjusting their trajectories to remain compatible with higher-priority ones. The implementation complies with Assumption 5 and adopts the same timestep, replanning time, and allocation-horizon parameters reported in Table 4, ensuring consistent evaluation conditions. Constraint reasoning is not supported by PBS, and a deadlock arises whenever no priority assignment can produce feasible trajectories. Deadlocks typically emerge in swap interactions or in narrow areas of the roadmap where path compatibility is structurally restricted. Countermeasures are introduced by assigning distinct goal nodes to active AGVs and by enforcing different priority relations for the first and second task composing each path. Separate priority relations increase the chance of obtaining a feasible priority structure but do not overcome the intrinsic limitations of priority-driven planning. Deadlocks occurring under PBS remain unresolved, because the deadlock detector and handler in the proposed traffic management system rely on CBS-style constraint reasoning, which cannot be translated into

the priority-only structure of PBS. As a consequence, the resolution computed by the deadlock detector and handler cannot be exploited by an L-MAPF coordinator that relies solely on priority relations. The limitation is not addressed by designing an alternative strategy for deadlock resolution, since incorporating a separate module tailored specifically to PBS would introduce behaviors and assumptions that diverge from the intended scope of the comparison. Our objective is to evaluate PBS as a pure priority-driven baseline within the L-MAPF coordinator, rather than proposing an alternative framework for traffic management.

To enable a fair comparison across the evaluated traffic management strategies, intervals during which AGVs remain blocked in undetected or unresolved deadlocks are excluded from the computation of throughput, management efficiency, and average flow time. Two of the three baselines do not incorporate deadlock detection or resolution, and including blockage intervals would impose penalties unrelated to the underlying coordination logic. Reported KPIs therefore reflect effective operating periods.

The comparative analysis shows clear advantages of the proposed TM in each scenario, as reported in Table 8.

In Layout 1, which features narrow corridors and strong spatial constraints, search-based conflict resolution provides a clear advantage, generating improvements in throughput of approximately 10% over the rule-based TM in 1.B.f, 7% over the state-of-the-art TM in 1.B.g, and 6% over the priority-based TM in 1.B.h. The results indicate that the proposed traffic management system outperforms both traditional and state-of-the-art rule-based coordination strategies, as rule-driven approaches typically impose rigid precedence relations that become inefficient in such constrained environments. In contrast, the proposed TM resolves conflicts with greater flexibility and prevents the formation of bottlenecks. The performance gap with the priority-based TM can be largely attributed to the specific geometry of Layout 1. Around the wrapping area, multiple AGVs must converge into a confined region, and fixed priority chains often become overly restrictive, producing extended blocking sequences and persistent congestion. Deadlock behavior reinforces the same trend: every deadlock occurring in 1.B.b is detected and resolved, whereas the rule-based TM and the priority-based TM leave all encountered deadlocks unresolved, and the state-of-the-art TM resolves only two of the three detected cases. The combination of effective conflict resolution and reliable deadlock recovery proves critical in a constrained environment where small disturbances rapidly propagate through interconnected corridors.

In Layout 2, the increase in traffic density and the presence of heterogeneous AGV classes amplify the differences among the evaluated coordination strategies. The rule-based TM in 2.C.f and the priority-based TM in 2.C.h tend to accumulate substantial idle time, especially when vehicles converge on shared areas and intersections. In these conditions, neither handcrafted rules nor priority relations are able to regulate high density AGV traffic, leading to persistent waiting times and reduced system reactivity. The state-of-the-art TM in 2.C.g alleviates part of this behavior thanks to its predictive components and simple negotiation rules, although deadlocks and long waiting periods still emerge frequently under dense operating conditions. The proposed TM maintains a clearly superior operating profile in this layout. Through ABH-CBS, which incorporates the anytime and EHC strategies, bottlenecks are minimized, and the deadlock-handling mechanism resolves nearly all blocking situations. As a result, throughput increases by roughly 11% compared to the rule-based TM, 10% compared to the priority-based TM, and by about 7% compared to the state-of-the-art TM, while average flow time remains consistently lower. All the baseline strategies leave several AGVs in long-term blocking conditions, underscoring the relevance of explicit and effective deadlock management and coordinated trajectory adaptation in medium-sized facilities.

In the large and regular geometry of Layout 3, where unidirectional tracks dominate and interaction density remains low, all coordination strategies achieve similar throughput in 3.C.f, 3.C.g and 3.C.h, reflecting the intrinsically limited complexity of the environment. Even in this favorable setting, the proposed TM preserves a slight performance edge. Flow time remains marginally lower, and the system resolves the only deadlock encountered during the evaluation. The rule-based TM and the priority-based TM both experience blocking situations that cannot be recovered, generating one and two unresolved deadlocks respectively. The state-of-the-art TM successfully resolves its single deadlock but still exhibits a modest increase in flow time compared to the proposed solution.

10.2.5 Convergence Analysis To empirically assess the convergence properties of ABH-CBS toward the optimal and complete CBS solution, the evolution of the solution cost is evaluated as the time horizon δ increases. ABH-CBS functions as a bounded-horizon planner whose window is selectively extended within corridor sectors through the EHC mechanism. As δ grows, the planner refines the partial solution in non-corridor regions; once δ becomes sufficiently large to cover the full execution horizon of all AGVs, ABH-CBS coincides with the full-horizon formulation of CBS and returns the optimal complete solution.

To examine this behavior in a realistic operational setting, approximately one hour of continuous operation of scenario 1.B is analyzed, yielding 3982 ABH-CBS planning instances. In each instance, ABH-CBS is executed without a timeout, allowing the bounded horizon to expand arbitrarily. During the update of the ABH-CBS solution $\mathcal{H}^\star$ in Algorithm 4, we record the node cost $\sum_{\gamma^a \in \mathcal{H}^\star} |\gamma^a|(\delta)$ for every attained value of δ. Each cost value is then compared with the CBS reference cost $\sum_{\gamma^a \in \mathcal{H}^{\text{CBS}}} |\gamma^a|$, defined as the cost of the CBS solution $\mathcal{H}^{\text{CBS}}$ computed with the same input configuration of the corresponding instance. The comparison is expressed through a *Normalized Cost Gap (NCG)*, defined as the relative difference between the ABH-CBS cost of the collision-free trajectory at a given δ and the full CBS cost:

$$\text{NCG}(\delta) = \frac{\sum_{\gamma^a \in \mathcal{H}^\star} |\gamma^a|(\delta) - \sum_{\gamma^a \in \mathcal{H}^{\text{CBS}}} |\gamma^a|}{\sum_{\gamma^a \in \mathcal{H}^{\text{CBS}}} |\gamma^a|}.$$

Figure 13 reports the convergence curves of the normalized cost gap toward the optimal and complete CBS solution as the time horizon δ increases. Thin colored lines correspond to 50 randomly selected instances,

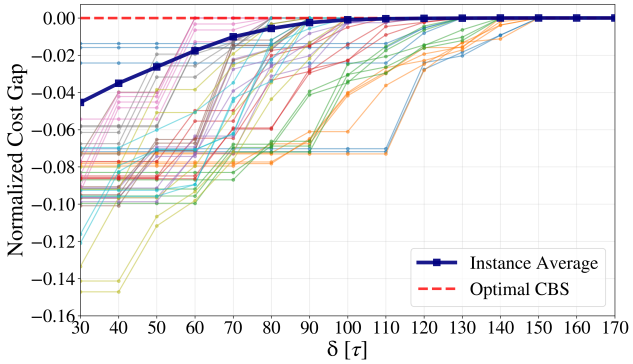


Figure 13. Convergence of ABH-CBS toward the full-horizon CBS solution. Thin colored curves correspond to 50 randomly selected instances, while the thick blue curve reports the average normalized cost gap over all 3982 instances of scenario 1.B.

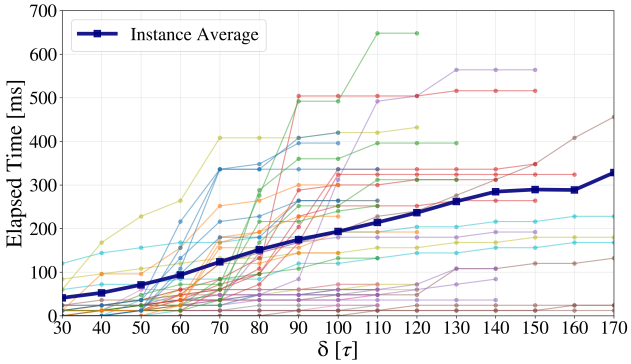


Figure 14. Execution time of ABH-CBS in relation to the bounded-horizon δ. Thin colored lines correspond to 50 randomly selected instances. The thick blue curve reports the average elapsed time computed over all 3982 instances in scenario 1.B for each value of the bounded-horizon parameter.

while the thick blue line shows the average behavior over all 3982 instances. A clear pattern emerges: as δ grows, the normalized cost gap increases monotonically and eventually approaches zero, demonstrating convergence of ABH-CBS toward the optimal CBS solution as the time window expands. The bounded-horizon formulation generally provides costs lower than the full CBS cost because, once the window is exceeded, additional conflicts are no longer detected and no further delays are inserted into the partial plan. The observed convergence reflects the design principle underlying ABH-CBS: when operating under real-time constraints, the bounded-horizon formulation focuses computation on resolving imminent conflicts and delivers high-quality solutions within the imposed timeout; as the horizon grows, e.g., in offline or relaxed-time conditions, the search progressively recovers the full CBS structure and converges to the optimal, complete solution.

In addition to evaluating convergence in terms of solution cost, the computational profile of ABH-CBS is examined by monitoring execution time as the time horizon δ increases. For each planning instance, a timer is reset at the start of the ABH-CBS run, and the elapsed time is recorded whenever $\mathcal{H}^\star$ is updated during the progressive expansion of the bounded horizon in Algorithm 4. The procedure yields, for every attained value of δ, a corresponding execution time that reflects the cumulative computation required to refine the partial plan up to that horizon.

Figure 14 reports the resulting execution-time curves. Thin colored lines correspond to 50 randomly selected instances and show the variability of the solver's computation time as a function of the time horizon δ. The thick blue curve presents the average elapsed time computed over all 3982 instances for each value of the time horizon.

For all reported instances, ABH-CBS computes an initial solution for the base time horizon δ' within $t^\sigma = 250$ ms. Consequently, every run reaches at least the base horizon inside the allowed timeout, which directly accounts for the 100% valid-solution rate observed in the scalability analysis for scenario 1.B. The average elapsed time grows approximately linearly with the time horizon δ, reflecting the incremental computation required to refine the plan as the bounded window expands. At the same time, execution times vary substantially across instances: some configurations are resolved within a few milliseconds, whereas others exceed $600, \mathrm{ms}$ due to higher conflict density and a correspondingly larger branching effort. The spread in computation times illustrates the dependence of the computational load on the underlying multi-agent interaction pattern, while the overall distribution confirms that ABH-CBS remains compatible with real-time operation.

10.3 Findings

The results obtained from the above analyses offer a unified picture of how the proposed system behaves across different operating conditions.

The proposed traffic management system effectively coordinates a heterogeneous AGV fleet on non-standardized industrial roadmaps with narrow bidirectional corridors and reduced maneuvering space, while also delivering consistent performance in more regular and spacious layouts.

Throughput grows with fleet size until each layout reaches the saturation point imposed by geometry and interaction density. Beyond that limit, additional AGVs no longer enhance productivity, although the L-MAPF coordinator continues to handle conflicts within real-time bounds except in the most congested configurations. The allocation-horizon parameter exerts limited influence on system-level performance: a broad range of values yields comparable results, with short horizons working best in irregular layouts and longer horizons remaining advantageous in more structured environments.

The ablation study indicates that both the anytime mechanism and the EHC strategy play a critical role in complex layouts. Disabling either one delays conflict detection, increases deadlock occurrences, and degrades throughput, flow time, and management efficiency. The effect is weaker in large, regular environments but remains beneficial whenever interaction density rises.

The comparative evaluation shows that the proposed system surpasses the rule-based TM, the state-of-the-art TM, and the priority-based L-MAPF variant across all non-standardized layouts. Anticipation of conflicts, coordinated motion under high density, and reliable deadlock detection and resolution produce consistently higher throughput and lower travel times in geometrically constrained settings.

The convergence analysis confirms that the ABH-CBS solver approaches the optimal and complete CBS solution as the time horizon expands. Execution-time measurements further show that real-time constraints are met across all evaluated instances while preserving the ability to recover the full CBS solution when additional computation is available.

11 Conclusions and Outlooks

In this paper, we present an innovative methodology for efficiently managing high-density AGV traffic in complex industrial environments. By employing a Lifelong Multi-Agent Path Finding strategy, our approach ensures robust and scalable coordination while maintaining locally optimal performance. Furthermore, our system guarantees safety through the path allocator module compliant with the VDA5050 standard and incorporates real-time deadlock resolution using an enhanced Conflict-Based Search algorithm. Extensive validation in real-world layouts shows throughput improvements of up to 11%, together with enhanced operational continuity and coordination efficiency, when compared with a conventional rule-based system, a state-of-the-art industrial solution, and an alternative L-MAPF method. Improvements are demonstrated in non-standardized settings involving large and heterogeneous AGVs operating on a roadmap, and the results indicate that the proposed traffic management system generalizes effectively to larger, less constrained environments while retaining real-time performance.

Future work will focus on relaxing the assumption that AGVs must follow the paths established during task assignment. A first direction is the introduction of real-time adaptive path adjustments to alleviate congestion and reduce conflict frequency during execution. Another area of extension concerns online replanning to navigate around obstacles, since the current fixed-path constraint may otherwise result in prolonged waiting. Further research will also investigate learning-based coordination strategies capable of reproducing the behavior of the proposed system, with the aim of improving scalability and robustness in large and uncertain industrial environments.

12 Acknowledgments

The authors gratefully acknowledge the *Gruppo Tecno-Ferrari S.p.A.*, and particularly Anna Bellodi, Riccardo Benedetti, and Andrea Bargi from the Software Development Office, for their support in the preparation of this paper.

13 Declaration of Conflicting Interests

The author(s) declared no potential conflicts of interest with respect to the research, authorship, and/or publication of this article.

14 Funding

The author(s) disclosed receipt of the following financial support for the research, authorship, and/or publication of this article: This work was supported by the Socially-acceptable Extended Reality Models and Systems (SERMAS) Project of the European Union's Horizon Europe Research and Innovation Program (GA n. 101070351).

15 ORCID iDs

Alessandro Bonetti https://orcid.org/0009-0008-2068-6165
Silvia Proia https://orcid.org/0000-0002-3801-4745
Simone Guidetti https://orcid.org/0009-0002-1446-1142
Lorenzo Sabattini https://orcid.org/0000-0002-2734-5549

References

Andreychuk A, Yakovlev K, Surynek P, Atzmon D and Stern R (2022) Multi-agent pathfinding with continuous time. *Artificial Intelligence* 305: 103662.

Azadeh K, De Koster R and Roy D (2019) Robotized and automated warehouse systems: Review and recent developments. *Transportation Science* 53(4): 917–945.

Azimi P, Haleh H and Mehran A (2010) The selection of the best control rule for a multiple-load agv system using simulation and fuzzy madm in a flexible manufacturing system. *Modelling and Simulation in Engineering* 2010.

Berndt M, Krummacker D, Fischer C and Schotten HD (2021) Centralized robotic fleet coordination and control. In: *Mobile Communication - Technologies and Applications; 25th ITG-Symposium*. pp. 1–8.

Bondy JA and Murty USR (2008) *Graph theory*. Springer Publishing Company, Incorporated.

Bonetti A, Guidetti S and Sabattini L (2024a) Efficient deadlock detection and resolution algorithm for agv fleet management. In: Secchi C and Marconi L (eds.) *European Robotics Forum 2024*. Cham: Springer Nature Switzerland. ISBN 978-3-031-76428-8, pp. 269–274.

Bonetti A, Proia S, Guidetti S and Sabattini L (2024b) Agv traffic management in automated industrial plants: An enhanced lifelong multi-agent path finding approach. In: *2024 IEEE 20th International Conference on Automation Science and Engineering (CASE)*. IEEE, pp. 626–632.

Chen M, Mangalathu S and Jeon JS (2023a) Betweenness centrality-based seismic risk management for bridge transportation networks. *Engineering Structures* 289: 116301.

Chen X, Xing Z, Feng L, Zhang T, Wu W and Hu R (2023b) An etcen-based motion coordination strategy avoiding active and passive deadlocks for multi-agv system. *IEEE Transactions on Automation Science and Engineering* 20(2): 1364–1377.

Cormen TH, Leiserson CE, Rivest RL and Stein C (2001) *Introduction to Algorithms*. 2nd edition. The MIT Press. ISBN 0262032937.

Damani M, Luo Z, Wenzel E and Sartoretti G (2021) Primal _2: Pathfinding via reinforcement and imitation multi-agent learning-lifelong. *IEEE Robotics and Automation Letters* 6(2): 2666–2673.

De Ryck M, Versteyhe M and Debrouwere F (2020) Automated guided vehicle systems, state-of-the-art control algorithms and techniques. *Journal of Manufacturing Systems* 54: 152–173.

Dergachev S and Yakovlev K (2021) Distributed multi-agent navigation based on reciprocal collision avoidance and locally confined multi-agent path finding. In: *2021 IEEE*

17th International Conference on Automation Science and Engineering (CASE). IEEE, pp. 1489–1494.

Digani V, Sabattini L, Secchi C and Fantuzzi C (2015) Ensemble coordination approach in multi-agv systems applied to industrial warehouses. *IEEE Transactions on Automation Science and Engineering* 12(3): 922–934.

Edelkamp S and Schrödl S (2012) Chapter 2–basic search algorithms. *Heuristic search* : 47–87.

Ericson C (2004) *Real-Time Collision Detection*. USA: CRC Press, Inc. ISBN 1558607323.

Gao J, Li Y, Li X, Yan K, Lin K and Wu X (2023) A review of graph-based multi-agent pathfinding solvers: From classical to beyond classical. *Knowledge-Based Systems* : 111121.

Gordon O, Filmus Y and Salzman O (2021) Revisiting the complexity analysis of conflict-based search: New computational techniques and improved bounds. In: *Proceedings of the International Symposium on Combinatorial Search*, volume 12. pp. 64–72.

Grenouilleau F, Van Hoeve WJ and Hooker JN (2019a) A multi-label a* algorithm for multi-agent pathfinding. In: *Proceedings of the international conference on automated planning and scheduling*, volume 29. pp. 181–185.

Grenouilleau F, Van Hoeve WJ and Hooker JN (2019b) A multi-label a* algorithm for multi-agent pathfinding. In: *Proceedings of the International Conference on Automated Planning and Scheduling*, volume 29. pp. 181–185.

Hönig W, Kumar T, Cohen L, Ma H, Xu H, Ayanian N and Koenig S (2016) Multi-agent path finding with kinematic constraints. In: *Proceedings of the International Conference on Automated Planning and Scheduling*, volume 26. pp. 477–485.

Hu H, Jia X, Liu K and Sun B (2021) Self-adaptive traffic control model with behavior trees and reinforcement learning for agv in industry 4.0. *IEEE Transactions on Industrial Informatics* 17(12): 7968–7979.

Hönig W, Kiesel S, Tinka A, Durham JW and Ayanian N (2019) Persistent and robust execution of mapf schedules in warehouses. *IEEE Robotics and Automation Letters* 4(2): 1125–1131.

Li J, Tinka A, Kiesel S, Durham JW, Kumar TS and Koenig S (2021) Lifelong multi-agent path finding in large-scale warehouses. In: *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 35. pp. 11272–11281.

Liu X, Xu Y, Cao J, Liu J and Zhao Y (2023) A high-precision planar nurbs interpolation system based on segmentation method for industrial robot. *Applied Sciences* 13(24): 13210.

Liu Z, Wang H, Wei H, Liu M and Liu YH (2021) Prediction, planning, and coordination of thousand-warehousing-robot networks with motion and communication uncertainties. *IEEE Transactions on Automation Science and Engineering* 18(4): 1705–1717.

Ma H, Harabor D, Stuckey PJ, Li J and Koenig S (2019) Searching with consistent prioritization for multi-agent path finding. In: *Proceedings of the AAAI conference on artificial intelligence*, volume 33. pp. 7643–7650.

Ma H, Li J, Kumar TS and Koenig S (2017) Lifelong multi-agent path finding for online pickup and delivery tasks. In: *Proceedings of the 16th Conference on Autonomous Agents and MultiAgent Systems*, AAMAS '17. Richland, SC: International Foundation for Autonomous Agents and Multiagent Systems, p. 837–845.

Madar N, Solovey K and Salzman O (2022) Leveraging experience in lifelong multi-agent pathfinding. In: *Proceedings of the International Symposium on Combinatorial Search*, volume 15. pp. 118–126.

Małopolski W and Zajac J (2021) Agvs collision and deadlock handling based on structural online control policy: A case study in a square topology. *Applied Sciences* 11(14): 6494.

Menniti S, Castagna E and Mazza T (2013) Estimating the global density of graphs by a sparseness index. *Applied Mathematics and Computation* 224: 346–357.

Okumura K, Machida M, Défago X and Tamura Y (2022) Priority inheritance with backtracking for iterative multi-agent path finding. *Artificial Intelligence* 310: 103752.

Peters J (2002) Geometric continuity.

Pratissoli F, Brugioni R, Battilani N and Sabattini L (2023) Hierarchical traffic management of multi-agv systems with deadlock prevention applied to industrial environments. *IEEE Transactions on Automation Science and Engineering* : 1–15.

Proia S, Cavone G, Camposeo A, Ceglie F, Carli R and Dotoli M (2022) Safe and ergonomic human-drone interaction in warehouses. In: *2022 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, pp. 6681–6686.

Ren Z, Nandy A, Rathinam S and Choset H (2024) Dms*: Towards minimizing makespan for multi-agent combinatorial path finding. *IEEE Robotics and Automation Letters* 9(9): 7987–7994.

Sabattini L, Digani V, Lucchi M, Secchi C and Fantuzzi C (2015) Mission assignment for multi-vehicle systems in industrial environments. *IFAC-PapersOnLine* 48(19): 268–273.

Santos J, Rebelo PM, Rocha LF, Costa P and Veiga G (2021) A* based routing and scheduling modules for multiple agvs in an industrial scenario. *Robotics* 10(2): 72.

Sharon G, Stern R, Felner A and Sturtevant NR (2015) Conflict-based search for optimal multi-agent pathfinding. *Artificial Intelligence* 219: 40–66.

Siegfried P and Bourafa R (2023) A review of the automated guided vehicle systems: dispatching systems and navigation concept. *Automobile Transport* : 2023–629.

Silver D (2005) Cooperative pathfinding. In: *Proceedings of the aaai conference on artificial intelligence and interactive digital entertainment*, volume 1. pp. 117–122.

Song S, Na KI and Yu W (2023) Anytime lifelong multi-agent pathfinding in topological maps. *IEEE Access* 11: 20365–20380.

Stern R, Sturtevant NR, Felner A, Koenig S, Ma H, Walker TT, Li J, Atzmon D, Cohen L, Kumar TKS, Boyarski E and Bartak R (2019) Multi-agent pathfinding: Definitions, variants, and benchmarks. *Symposium on Combinatorial Search (SoCS)* : 151–158.

Tien TN and Nguyen KV (2022) Updated weight graph for dynamic path planning of multi-agvs in healthcare environments. In: *2022 International Conference on Advanced Technologies for Communications (ATC)*. pp. 130–135.

Venkatesh S, Smith J, Deuermeyer B and Curry G (1998) Deadlock detection and resolution for discrete-event simulation: multiple-unit seizes. *IIE transactions* 30(3): 201–216.

Verma P, Olm JM and Suarez R (2024) Traffic management of multi-agv systems by improved dynamic resource reservation. *IEEE Access* 12: 19790–19805.

Wagner G and Choset H (2015) Subdimensional expansion for multirobot path planning. *Artificial Intelligence* 219: 1–24.

Winkelhaus S and Grosse EH (2020) Logistics 4.0: a systematic review towards a new logistics system. *International Journal of Production Research* 58(1): 18–43.

Xie W, Peng X, Liu Y, Zeng J, Li L and Eisaka T (2024) Conflict-free coordination planning for multiple automated guided vehicles in an intelligent warehousing system. *Simulation Modelling Practice and Theory* 134: 102945.

Xu Q, Li J, Koenig S and Ma H (2022) Multi-goal multi-agent pickup and delivery. In: *2022 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. pp. 9964–9971.

Yu J and LaValle S (2013) Structure and intractability of optimal multi-robot path planning on graphs. In: *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 27. pp. 1443–1449.

Zhang K, Mao J, Zhang S and Wang N (2024) A priority-based hierarchical framework for k-robust multi-agent path finding. *IEEE Transactions on Intelligent Vehicles* .

Zhang Z, Han R and Pan J (2022) An efficient centralized planner for multiple automated guided vehicles at the crossroad of polynomial curves. *IEEE Robotics and Automation Letters* 7(1): 398–405.

Zhou Y and Suri S (2000) Collision detection using bounding boxes: Convexity helps. In: *European Symposium on Algorithms*. Springer, pp. 437–448.

Ziegler J and Stiller C (2010) Fast collision checking for intelligent vehicle motion planning. In: *2010 IEEE Intelligent Vehicles Symposium*. pp. 518–522.

A Low-Level Planner

As detailed in Algorithm 7 - Low-Level Planner, our space-time A* takes as input the current target vertex v_z^a, the corresponding target timestep τ_z^a, the next goal vertex $v^{g',a}$, the path subgraph $\mathcal{K}^a$, the constraint set $\mathcal{M}$ defined by the high-level planner, and the static and dynamic obstacle sets $\mathcal{O}^s$,$\mathcal{U}^{\setminus a}$. Algorithm 7 is initialized with a start node (v_z^a, τ_z^a, f_z^a), where the total cost f_z^a is given by the sum of τ_z^a and the heuristic value $h(v_z^a, v^{g',a})$. Specifically, the heuristic $h(v_i^a, v^{g',a})$ estimates the cost to reach $v^{g',a}$ from a generic vertex $v_i^a \in \pi^a$ and is defined as:

$$h(v_i^a, v^{g',a}) = \sum_{m=i}^{L^a-1} D^a_{m,m+1} \tag{14}$$

where $D^a_{m,m+1}$ represents the timestep duration of traversing the edge $e^a_{m,m+1}$. The start node is inserted into a list, referred to as the OPEN_L list, which contains the nodes to be explored (Algorithm 7, line 1). At this step, Algorithm 7 initializes another list, referred to as the CLOSED_L list, which is used to prevent redundant expansions by storing processed nodes (Algorithm 7, line 2). In addition, a dictionary named *traceback* is introduced to facilitate trajectory reconstruction upon reaching the goal vertex $v^{g',a}$ (Algorithm 7, line 3).

During execution, while OPEN_L list is not empty, Algorithm 7 selects the current node $(v_i^a, \tau^{\rho,a}, f^{\rho,a})$ with the lowest value of $f^{\rho,a}$ from the OPEN_L list for expansion (Algorithm 7, line 5). If v_i^a corresponds to the goal vertex $v^{g',a}$, Algorithm 7 terminates successfully invoking the *GetTrajectory* function, which reconstructs the trajectory γ^a by backtracking through the *traceback* dictionary from $v^{g',a}$ to v_z^a (Algorithm 7, lines 6-7). Otherwise, Algorithm 7 removes the current node from OPEN_L list, adds it to the CLOSED_L list, and retrieves the neighbour vertices v_j^a, with $j = i \vee i+1$, by using the *GetNeighbors* function applied to the path subgraph $\mathcal{K}^a$ (Algorithm 7, lines 8-10).

For each neighbour vertex v_j^a, Algorithm 7 calculates the corresponding action $\{v_i^a, v_j^a, \tau^{\rho,a}, D^a_{i,j}\}$, the next timestep $\tau^{\zeta,a}$, and the cost $f^{\zeta,a}$ using the heuristic h (Algorithm 7, lines 11-13). Algorithm 7 first checks whether the neighbour node $(v_j^a, \tau^{\zeta,a}, f^{\zeta,a})$ is in the CLOSED_L list, indicating that it has already been expanded in the previous iterations (Algorithm 7, line 14). If this condition is not verified, Algorithm 7 proceeds to validate the action $\{v_i^a, v_j^a, \tau^{\rho,a}, D^a_{i,j}\}$ against the constraint set $\mathcal{M}$, using the *CheckConstraints* function (Algorithm 7, line 16). If the action satisfies the constraints, Algorithm 7 then verifies its safety by checking for collisions with static or dynamic obstacles using Algorithm 8 - Check Obstacles Collision (Algorithm 7, line 18).

Specifically, Algorithm 8 is employed to verify whether the move or wait action (see Section 6) from the vertex v_i^a to its neighbour v_j^a, where $j = i \vee i+1$, avoids static and dynamic obstacles. First, if the considered action is a move, Algorithm 8 checks if the corresponding edge $e^a_{i,i+1}$ belongs to the set of static obstacles $\mathcal{O}^s$ (Algorithm 8, lines 1-3). If this condition is not satisfied, the action can be safely executed without colliding with static obstructions. The

Algorithm 7: Low-Level Planner

Input: $v_z^a, \tau_z^a, v^{g',a}, \mathcal{K}^a, \mathcal{O}^s, \mathcal{U}^{\setminus a}, \mathcal{M}$
Output: γ^a

1 OPEN_L $\leftarrow \{(v_z^a, \tau_z^a, f_z^a \leftarrow \tau_z^a + h(v_z^a, v^{g',a}))\}$
2 CLOSED_L $\leftarrow \{\}$
3 $traceback \leftarrow \{\}$
4 **while** OPEN_L $\neq \emptyset$ **do**
5 $(v_i^a, \tau^{\rho,a}, f^{\rho,a}) \leftarrow$ lowest $f^{\rho,a}$ cost node from OPEN_L
6 **if** $v_i^a = v^{g',a}$ **then**
7 **return** $GetTrajectory(traceback, v_i^a, \tau^{\rho,a})$
8 OPEN_L $\leftarrow$ OPEN_L $\setminus (v_i^a, \tau^{\rho,a}, f^{\rho,a})$
9 CLOSED_L $\leftarrow$ CLOSED_L $\cup (v_i^a, \tau^{\rho,a}, f^{\rho,a})$
10 **foreach** $v_j^a \in GetNeighbors(\mathcal{K}^a, v_i^a)$ **do**
11 $\{v_i^a, v_j^a, \tau^{\rho,a}, D^a_{i,j}\} \leftarrow GetAction(v_i^a, v_j^a, \tau^{\rho,a}, \mathcal{K}^a)$
12 $\tau^{\zeta,a} \leftarrow \tau^{\rho,a} + D^a_{i,j}$
13 $f^{\zeta,a} \leftarrow \tau^{\zeta,a} + h(v_j^a, v^{g',a})$
14 **if** $(v_j^a, \tau^{\zeta,a}, f^{\zeta,a}) \in$ CLOSED_L **then**
15 continue
16 **if** $CheckConstraints(\{v_i^a, v_j^a, \tau^{\rho,a}, D^a_{i,j}\}, \mathcal{M})$ **then**
17 continue
18 **if Algorithm 8**$(\{v_i^a, v_j^a, \tau^{\rho,a}, D^a_{i,j}\}, \mathcal{O}^s, \mathcal{U}^{\setminus a})$ **then**
19 continue
20 **if** $(v_j^a, \tau^{\zeta,a}, f^{\zeta,a}) \notin$ OPEN_L **then**
21 $traceback[(v_j^a, \tau^{\zeta,a})] \leftarrow (v_i^a, \tau^{\rho,a})$
22 OPEN_L $\leftarrow$ OPEN_L $\cup (v_j^a, \tau^{\zeta,a}, f^{\zeta,a})$
23 **return** $\varnothing$

Algorithm 8: Check Obstacles Collision

Input: $\{v_i^a, v_j^a, \tau^a, D_{i,j}^a\}, \mathcal{O}^s, \mathcal{U}^{\setminus a}$
Output: Boolean

1 **if** $v_i^a \neq v_j^a$ **then**
2 $e_{i,i+1}^a \leftarrow GetEdge(v_i^a, v_{i+1}^a)$
3 **if** $e_{i,i+1}^a \in \mathcal{O}^s$ **then**
4 **return** true
5 **foreach** $u^b \in \mathcal{U}^{\setminus a}$ **do**
6 **foreach** $\{v_m^b, v_{m+1}^b, \tau^b, D_{m,m+1}^b\} \in u^b$ **do**
7 **if** **Algorithm 9**$(\{v_i^a, v_j^a, \tau^a, D_{i,j}^a\}, \{v_m^b, v_{m+1}^b, \tau^b, D_{m,m+1}^b\})$ **then**
8 **return** true
9 **return** false

Algorithm 9: Check Space-Time Collision

Input: $\{v_i^a, v_j^a, \tau^a, D_{i,j}^a\}, \{v_m^b, v_n^b, \tau^b, D_{m,n}^b\}$
Output: Boolean

1 **if** $\max(\tau^a, \tau^b) \leq \min(\tau^a + D_{m,n}^a, \tau^b + D_{i,j}^b)$ **then**
2 **if** $v_i^a = v_j^a$ **then**
3 $\mathcal{D}_i^a \leftarrow GetCollisionSet(v_i^a)$
4 **if** $v_m^b = v_n^b$ **then**
5 **if** $v_m^b \in \mathcal{D}_i^a$ **then**
6 **return** true
7 **else**
8 $e_{m,m+1}^b \leftarrow GetEdge(v_m^b, v_{m+1}^b)$
9 **if** $e_{m,m+1}^b \in \mathcal{D}_i^a$ **then**
10 **return** true
11 **else**
12 $\mathcal{D}_{i,i+1}^a \leftarrow GetCollisionSet(v_i^a, v_{i+1}^a)$
13 **if** $v_m^b = v_n^b$ **then**
14 **if** $v_m^b \in \mathcal{D}_{i,i+1}^a$ **then**
15 **return** true
16 **else**
17 $e_{m,m+1}^b \leftarrow GetEdge(v_m^b, v_{m+1}^b)$
18 **if** $e_{m,m+1}^b \in \mathcal{D}_{i,i+1}^a$ **then**
19 **return** true
20 **return** false

check then proceeds to dynamic obstacles. In particular, if the action $\{v_i^a, v_j^a, \tau^a, D_{i,j}^a\}$ results in a space-time collision with any other allocated trajectory $u_b \in \mathcal{U}^{\setminus a}$, Algorithm 8 asserts that the action is unsafe (Algorithm 8, lines 5-7).

The space-time collision check is performed using Algorithm 9 - Check Space-Time Collision, which evaluates whether the edges or vertices involved in two move or wait actions from different AGV trajectories are in collision with overlapping timestep intervals. Let $\{v_i^a, v_j^a, \tau^a, D_{i,j}^a\}$ be the action of the a-th AGV and let $\{v_m^b, v_n^b, \tau^b, D_{m,n}^b\}$ be the action of the b-th AGV, with $n = m \vee m+1$. Algorithm 9 first checks if the two actions have overlapping timestep intervals (Algorithm 9, line 1). Then, it retrieves the collision set $\mathcal{D}_i^a$ for the vertex v_i^a if $\{v_i^a, v_j^a, \tau^a, D_{i,j}^a\}$ is a wait action, or $\mathcal{D}_{i,i+1}^a$ for the edge $e_{i,i+1}^a$ if it is a move action (Algorithm 9, line 3 and line 12). Next, Algorithm 9 determines whether the vertex v_m^b or the edge $e_{m,m+1}^b$ is contained within the retrieved collision set (Algorithm 9, lines 4-10 and lines 13-19). If this condition is met, $\{v_i^a, v_j^a, \tau^a, D_{i,j}^a\}$ and $\{v_m^b, v_n^b, \tau^b, D_{m,n}^b\}$ are in space-time collision; otherwise, they are collision-free. Hence, if the action avoids static and dynamic obstacles, Algorithm 7 checks if the neighbour node $(v_j^a, \tau^{\zeta,a}, f^{\zeta,a})$ is absent from the OPEN_L list (Algorithm 7, line 20). If this condition holds, *traceback* is updated and $(v_j^a, \tau^{\zeta,a}, f^{\zeta,a})$ is added to OPEN_L list (Algorithm 7, lines 21-22). Algorithm 7 ultimately returns the trajectory γ^a for the a-th active AGV compliant with constraints and static and dynamic obstacles or an empty sequence if no such trajectory exists.

B Nomenclature

AGV Model

N^A	Number of AGVs
N^B	Number of AGV battery chargers
$\mathcal{A}$	Set of active AGVs
x	AGV x-coordinate
y	AGV y-coordinate
θ	AGV angle
$q = [x, y, \theta]^\top$	AGV pose
$\mathcal{C}$	Set of all AGV classes
C	AGV class
N^C	Number of AGV classes
ϕ	Convex polygonal footprint of C
$f(q, \dot{q}) = 0$	Kinematic constraints of C

Environment Model

$\mathcal{G}^R$	Roadmap layer graph
$\mathcal{V}^R$	Set of roadmap layer vertices
$\mathcal{E}^R$	Set of roadmap layer edges
p	Location
v	Vertex
N^V	Number of locations/vertices
r	Segment
e	Directed edge
N^E	Number of segments/edges
ω	Traversal time of edge e (edge weight)
D	Traversal timesteps duration
$\mathcal{G}^C$	Subgraph of roadmap layer relative to C
g	Generic roadmap element (vertex or edge)
$\mathcal{D}$	Collision set of g
$\mathcal{Y}$	Collision set computed with Algorithm 1
$\mathcal{J}$	Sampled poses for g
Φ	Projection of ϕ
$\mathcal{G}^{IS}$	Topological layer graph
$\mathcal{V}^{IS}$	Set of topological layer vertices
$\mathcal{V}^{IS}$	Set of topological layer edges
S	Sector
N^S	Number of sectors

Paths and Trajectories

v^s	Starting vertex
v^g	Goal vertex
$v^{g'}$	Next goal vertex
π^M	Main path from v_s to v_g
π^N	Extension path from v_g to $v^{g'}$
π	Fixed path

L^M	Length of π^M
L^N	Length of π^N
L	Length of π
Ω	Total traversal cost of π
γ	Trajectory
N^γ	Number of actions in γ

L-MAPF Coordinator

τ	Timestep
η	Replanning time
δ'	Base time horizon
δ''	Horizon increment
t^σ	Timeout
$\mathcal{V}^z$	Set of target vertices
$\mathcal{T}^z$	Set of target timesteps
$\mathcal{V}^{g'}$	Set of next goal vertices
$\mathcal{K}$	Path subgraph of π
$\mathcal{K}^{\text{tot}}$	Set of all path subgraphs
o	Static obstacle
N^O	Number of static obstacles
$\mathcal{O}$	Set of obstructed edges by o
$\mathcal{O}^s$	Set of static obstacles
$\mathcal{O}^d$	Set of dynamic obstacles
ξ	Extended corridor
$\mathcal{S}$	Set of extended corridors
J	Objective function (Sum of Costs)

ABH-CBS

t^ι	Initial computation time
λ	CBS node
$\mathcal{H}$	Set of trajectories
δ	Time horizon
$\mathcal{N}$	Set of horizons
μ	Constraint
$\mathcal{M}$	Set of constraints

Path Allocator

ϵ	Allocation horizon
w	Allocated queue
$\mathcal{W}$	Set of allocated queues
u	Allocated trajectory
$\mathcal{U}$	Set of allocated trajectories

Deadlock Detector and Handler

$\mathcal{G}^P$	Precedence graph
$\mathcal{V}^P$	Precedence vertex set
$\mathcal{E}^P$	Precedence edge set
$\mathcal{B}$	Set of AGVs involved in deadlock

Superscripts

a, b	AGV in $\mathcal{A}$
m, n	Elements in $\mathcal{G}^R$
b, d	Elements in $\mathcal{J}$
I	Initial value
F	Final value
s	Starting value
g	Goal value
z	Target item
r	Root for λ
p	Parent for λ
c	Child for λ
$\star$	Optimal value

Subscripts

i, j, m, n	Iteration numbers for π
k	Iteration number for γ, $\mathcal{C}$, $\mathcal{O}$, and $\mathcal{M}$
u	Iteration numbers for locations
h	Iteration numbers for segments
z	Target index for π
l	Currently occupied index for π